252-0027

Einführung in die Programmierung
Übungen

Woche 9: Klassen, Verlinkte Objekte

Jhalak Yadav
Departement Informatik
ETH Zürich

Organisatorisches

- Mein Name: Jhalak Yadav
- Bei Fragen: jyadav@student.ethz.ch
  - Mails bitte mit «[EProg26]» im Betreff
- Neue Aufgaben: **Dienstag Abend** (im Normalfall)
- Abgabe der Übungen bis **Dienstag Abend (23:59)** Folgewoche
  - Abgabe immer via Git
  - Lösungen in separatem Projekt auf Git

jhalakyadav.github.io

Probleme Lösen

Probleme Lösen: Labyrinth

Aufgabe 1: Labyrinth (2021 W8)

Ein Labyrinth besteht aus einer Menge von Räumen, welche durch die Klasse `Room` dargestellt werden. Die Klasse hat zwei Attribute: Der Integer `age` (grösser gleich 0) beschreibt das Alter des Raums und der Array `doorsTo` (nie `null`) beschreibt die Türen von diesem Raum zu anderen Räumen. Alle Türen sind Falltüren, d.h. sie funktionieren nur in eine Richtung. Ein Raum ist ein Ausgang aus dem Labyrinth, wenn keine Türen von dem Raum wegführen, das heisst, wenn `doorsTo` eine Länge von 0 hat.

Für alle Aufgaben werden Sie in einen zufälligen Raum geworfen, welcher als Argument gegeben wird (garantiert nicht `null`) und von welchem aus Sie die Aufgabe lösen müssen. Sie dürfen für alle Aufgaben annehmen, dass es im Labyrinth keinen Zyklus gibt. Das heisst, dass man einen Raum, welchen man durch eine Türe verlassen hat, nie wieder erreichen kann indem man weiteren Türen folgt. Eine Sequenz von N Räumen $r_1, \ldots, r_N$ ist ein *Lösungspfad* für einen Raum `room` genau dann wenn: (1) Der erste Raum r_1 ist der Raum `room`, (2) der letzte Raum r_N ist ein Ausgang, und (3) jeder Raum r_i mit $1 \leq i < N$ hat eine Tür zum nächsten Raum in der Sequenz r_{i+1}.

Probleme Lösen: Labyrinth

Ausgang wenn:

- `r.doorsTo.length == 0`

Lösungspfad $r_1, \ldots, r_N$ wenn:

- r_1 ist `room` (der Raum der uns gegeben wird)
- r_N ist ein Ausgang
- r_i und r_{i+1} sind jeweils durch eine Tür verbunden.

man weiteren Türen folgt. Eine Sequenz von N Räumen $r_1, \ldots, r_N$ ist ein *Lösungspfad* für einen Raum `room` genau dann wenn: (1) Der erste Raum r_1 ist der Raum `room`, (2) der letzte Raum r_N ist ein Ausgang, und (3) jeder Raum r_i mit $1 \leq i < N$ hat eine Tür zum nächsten Raum in der Sequenz r_{i+1}.

Probleme Lösen: Labyrinth

1. Implementieren Sie die Methode `Labyrinth.task1(Room room)`. Die Methode soll true zurückgeben genau dann, wenn es einen Lösungspfad $r_1, \ldots, r_N$ für `room` gibt, sodass:
   - Für jede Teilsequenz $r_1, ..., r_i$ mit $1 \leq i \leq N$ gilt, dass die Summe der Alter der Räume $r_1, ..., r_i$ nicht durch 3 teilbar ist.
2. Implementieren Sie die Methode `Labyrinth.task2(Room room)`. Die Methode soll true zurückgeben genau dann, wenn es zwei Lösungspfade $r_1, \ldots, r_N$ und $s_1, \ldots, s_N$ für `room` gibt, sodass:
   - Die Räume r_i und s_i haben das gleiche Alter für jedes i mit $1 \leq i \leq N$.
   - Für mindestens ein i mit $1 \leq i \leq N$ gilt, dass r_i und s_i unterschiedlich sind (verschiedene Referenzen).

Wichtig!

Sie dürfen Methoden und Felder der Klasse `Room` hinzufügen. Tests finden Sie in der Datei "LabyrinthTest.java". **Tipp:** Lösen Sie die Aufgaben rekursiv. Für keine der Aufgaben müssen Sie alle Pfade generieren und dann erst prüfen, dass die Eigenschaften gelten. Manche der Tests enthalten Labyrinthe mit einer extrem grossen Anzahl an Pfaden aber leichten Lösungen.

```java
public class Room {

    private int age;          // Age or the room, >= 0
    public Room[] doorsTo;    // Doors to other rooms, never null

    // PRE: age >= 0; doorsTo and all its elements are non-null.
    public Room(int age, Room... doorsTo) {
        this.age = age;
        this.doorsTo = doorsTo;
    }
    public boolean isExit() { // Exit if there are no doors
         return doorsTo.length == 0;
    }

    public int getAge() { // Return the room's age
        return age;
```

Was befindet sich in der `Labyrinth`-Klasse?

```java
public class Labyrinth {

    public static boolean task1(Room room) {
        // TODO
        return false;
    }

    public static boolean task2(Room room) {
        // TODO
        return false;
    }
}
```

Probleme Lösen: Labyrinth

1. Implementieren Sie die Methode `Labyrinth.task1(Room room)`. Die Methode soll true zurückgeben genau dann, wenn es einen Lösungspfad $r_1, \ldots, r_N$ für `room` gibt, sodass:
   - Für jede Teilsequenz $r_1, ..., r_i$ mit $1 \leq i \leq N$ gilt, dass die Summe der Alter der Räume $r_1, ..., r_i$ nicht durch 3 teilbar ist.

Wie lösen wir das Problem?

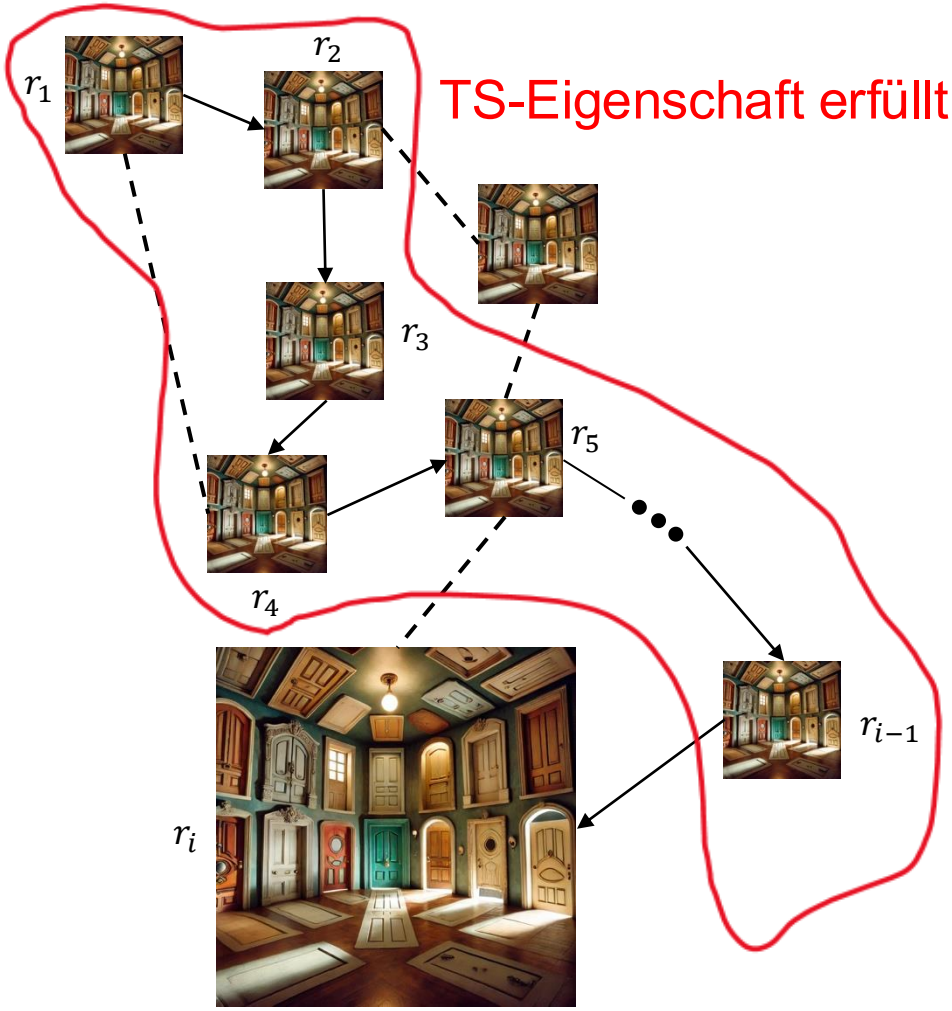
- **Start: ein Raum in einem gerichteten, zyklenfreien Labyrinth. Gesucht sind Wege zu einem Ausgang. Teil 1 prüft Präfixsummen, Teil 2 zwei verschiedene Wege mit gleicher Altersfolge.**



Teilprobleme identifizieren:

- Angenommen wir befinden uns in einem Raum r_i wie wissen wir ob der Pfad $r_1, \dots, r_i$ eine Lösung ist?

- **TS-Eigenschaft:** Für jeden Präfix ab dem Startraum ist die Summe der Raumalter nicht durch 3 teilbar. Beliebige Teilstücke im Inneren des Pfads sind nicht gemeint.



Teilprobleme identifizieren (Versuch 1):

Bisheriger Pfad: TS-Eigenschaft erfüllt.

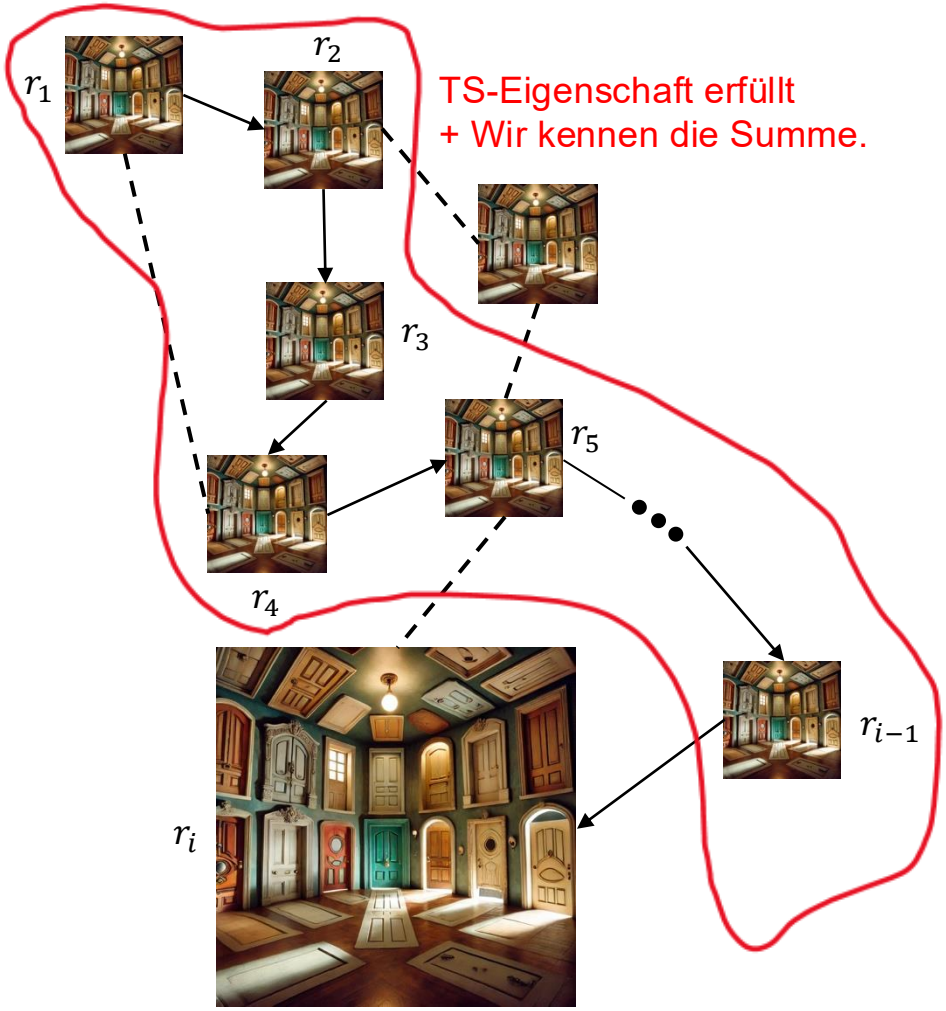
Reicht das, um einen Raum anzuhängen?

Nein: Bisherige Summe 22, nächstes Alter 11.

22 % 3 != 0

(22 + 11) % 3 == 0

Wir müssen auch die Summe kennen.



Teilprobleme identifizieren (Versuch 2):

Der bisherige Pfad erfüllt die TS-Eigenschaft.

sum enthält die Alterssumme dieses Pfads.

Für den nächsten Raum prüfen wir:

(sum + age) % 3 != 0

Nur dann setzen wir die Suche fort.

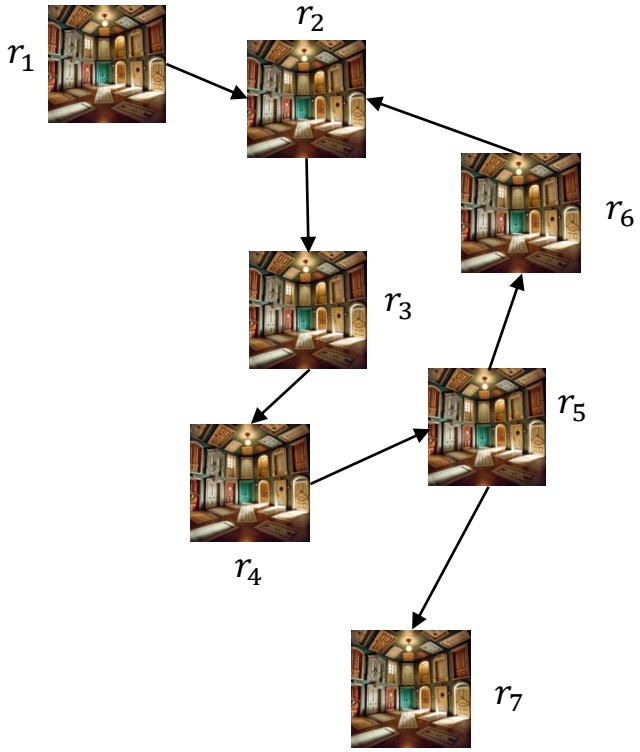
```java
// PRE: sum = Summe der bisherigen Räume; diese erfüllen die TS-Eigenschaft
public static boolean solve1(Room room, int sum) {
    sum = sum + room.getAge();

    if (sum % 3 == 0) { // TS-Eigenschaft verletzt: Pfad abbrechen
        return false;
    }
    if (room.isExit()) { // Ausgang erreicht: fertig
        return true;
    }
    for (int i = 0; i < room.doorsTo.length; i++) { // Alle Nachbarräume prüfen (keine Zyklen)
        if (solve1(room.doorsTo[i], sum)) { // Weg durch diese Tür führt zum Ausgang
            return true;
        }
    }
    return false; // Kein gültiger Weg zum Ausgang gefunden
}
```

Wie nutzen wir diese Methode nun?

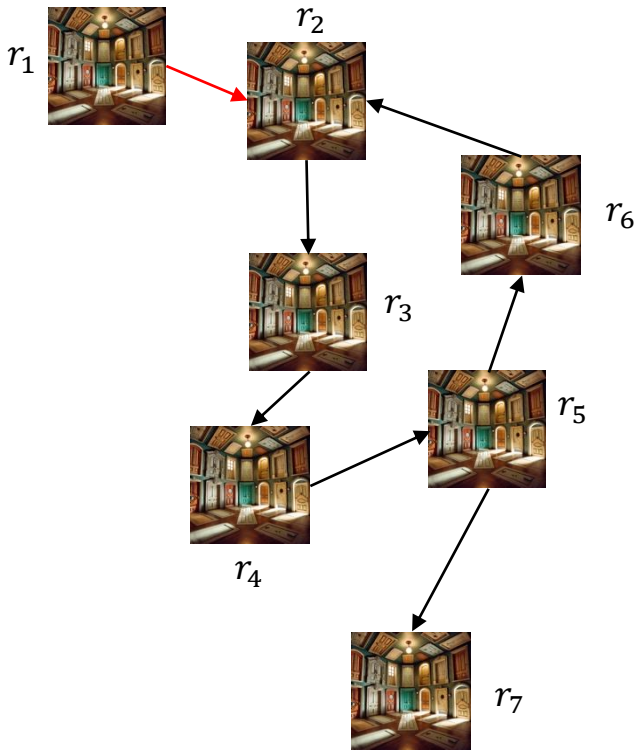
```java
public static boolean solve1(Room room, int sum) {
    // ...
}

public static boolean task1(Room room) {
    return solve1(room, 0); // Starte Suche bei room mit initialer Summe 0
}
```



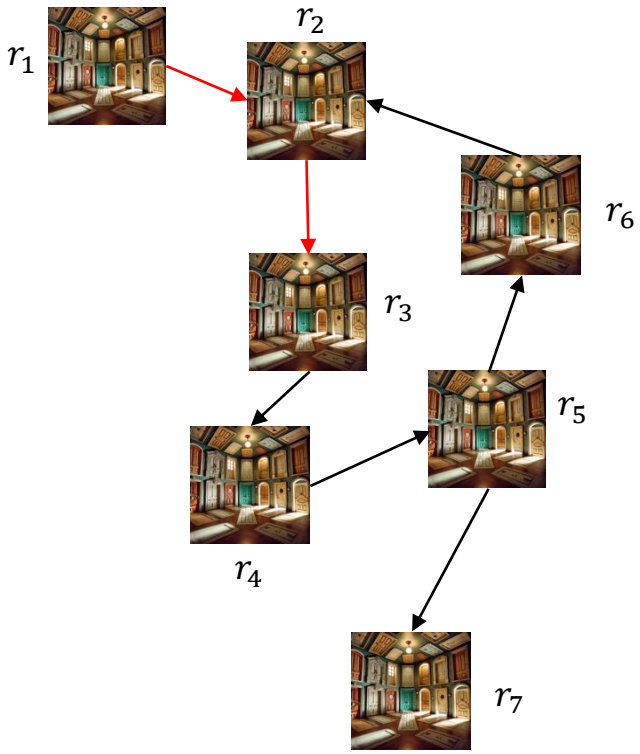
Wieso keine Zyklen?

- Wenn wir Zyklen haben kommt es zu **endloser Rekursion**, ausser wir merken uns explizit, in welchen Räumen wir bereits waren.



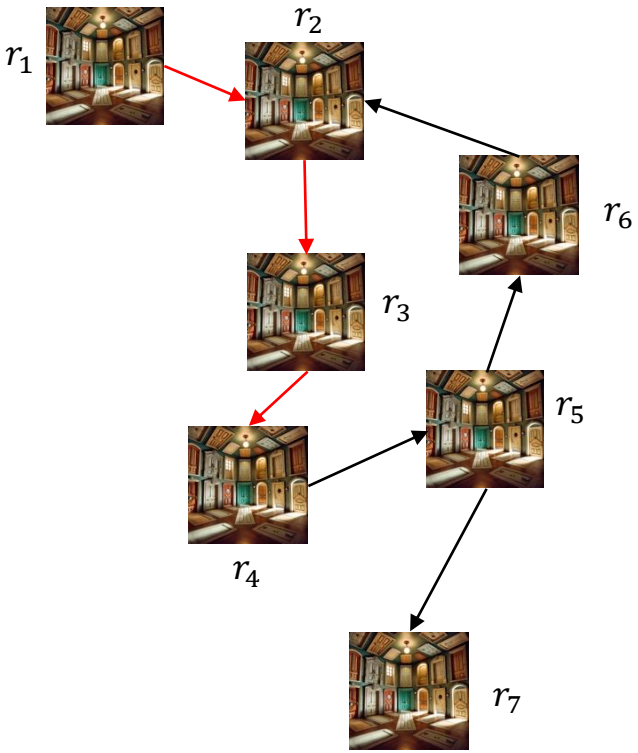
Wieso keine Zyklen?

- Wenn wir Zyklen haben kommt es zu **endloser Rekursion**, ausser wir merken uns explizit, in welchen Räumen wir bereits waren.



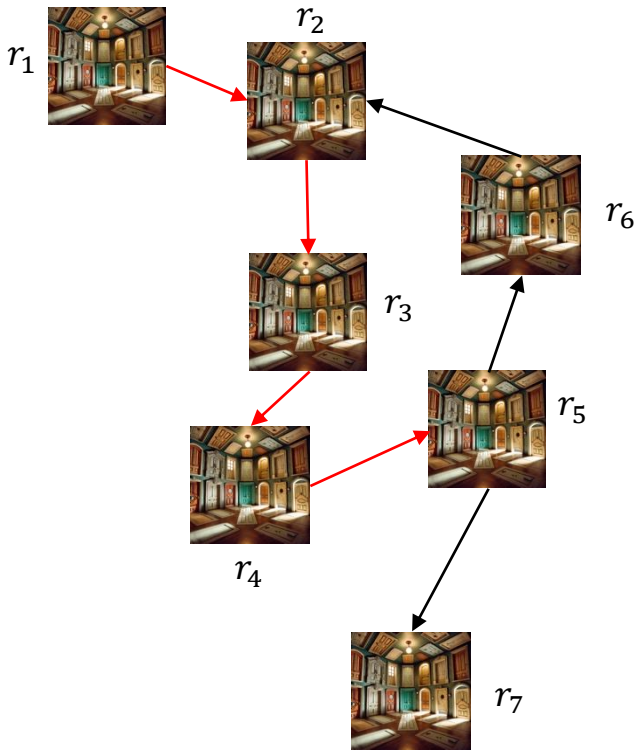
Wieso keine Zyklen?

- Wenn wir Zyklen haben kommt es zu **endloser Rekursion** ohne, dass wir uns merken, in welchen Räumen wir bereits waren.



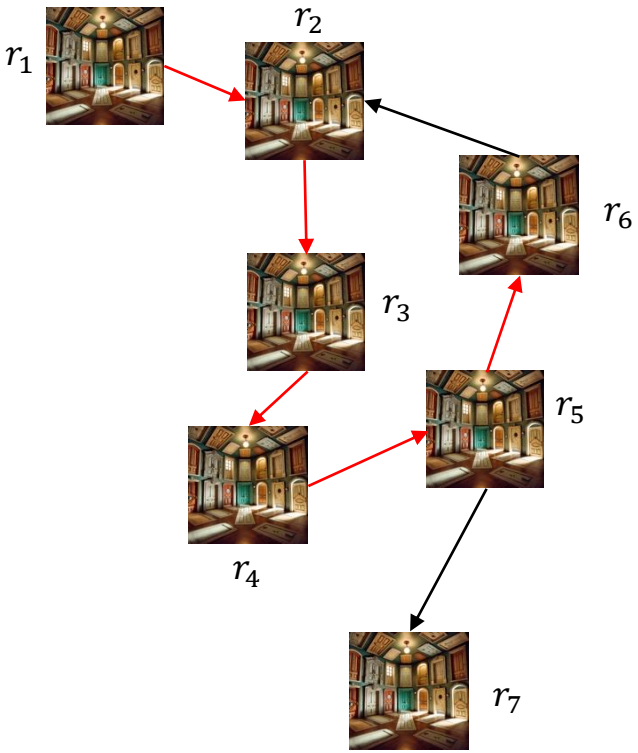
Wieso keine Zyklen?

- Wenn wir Zyklen haben kommt es zu **endloser Rekursion** ohne, dass wir uns merken, in welchen Räumen wir bereits waren.



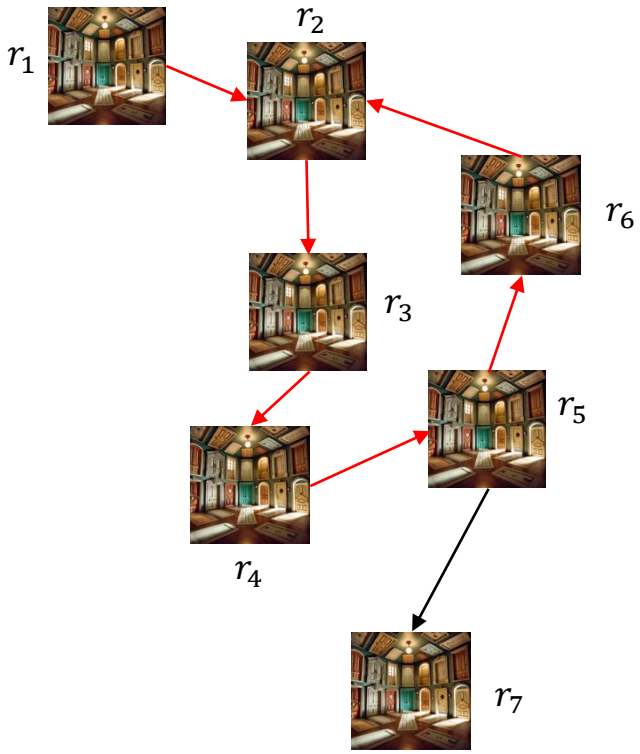
Wieso keine Zyklen?

- Wenn wir Zyklen haben kommt es zu **endloser Rekursion** ohne, dass wir uns merken, in welchen Räumen wir bereits waren.



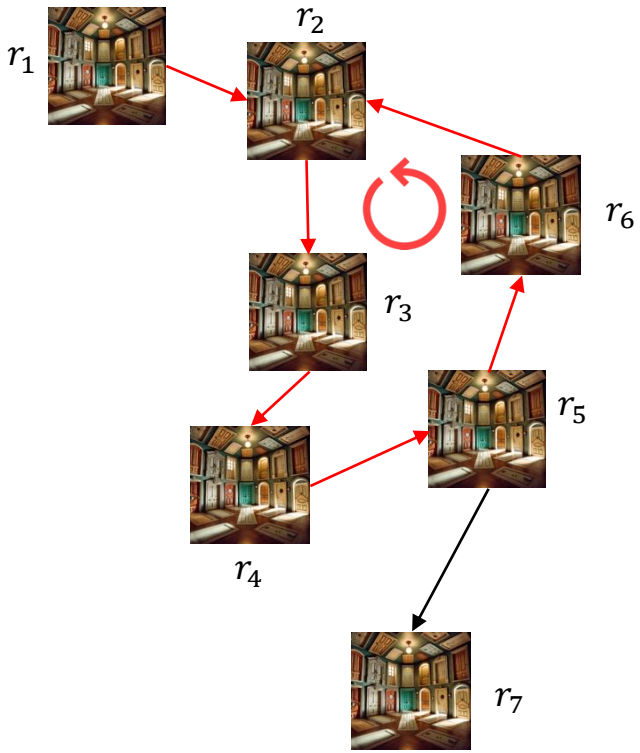
Wieso keine Zyklen?

- Wenn wir Zyklen haben kommt es zu **endloser Rekursion** ohne, dass wir uns merken, in welchen Räumen wir bereits waren.



Wieso keine Zyklen?

- Wenn wir Zyklen haben kommt es zu **endloser Rekursion** ohne, dass wir uns merken, in welchen Räumen wir bereits waren.



Wieso keine Zyklen?

- Wenn wir Zyklen haben kommt es zu **endloser Rekursion** ohne, dass wir uns merken, in welchen Räumen wir bereits waren.

Probleme Lösen: Labyrinth

2. Implementieren Sie die Methode `Labyrinth.task2(Room room)`. Die Methode soll true zurückgeben genau dann, wenn es zwei Lösungspfade $r_1, \ldots, r_N$ und $s_1, \ldots, s_N$ für `room` gibt, sodass:

   - Die Räume r_i und s_i haben das gleiche Alter für jedes i mit $1 \leq i \leq N$.
   - Für mindestens ein i mit $1 \leq i \leq N$ gilt, dass r_i und s_i unterschiedlich sind (verschiedene Referenzen).

Wie lösen wir das Problem?

- **Rekursive Lösung:** Damit wir das Problem rekursiv lösen können, müssen wir **Teilprobleme** identifizieren.

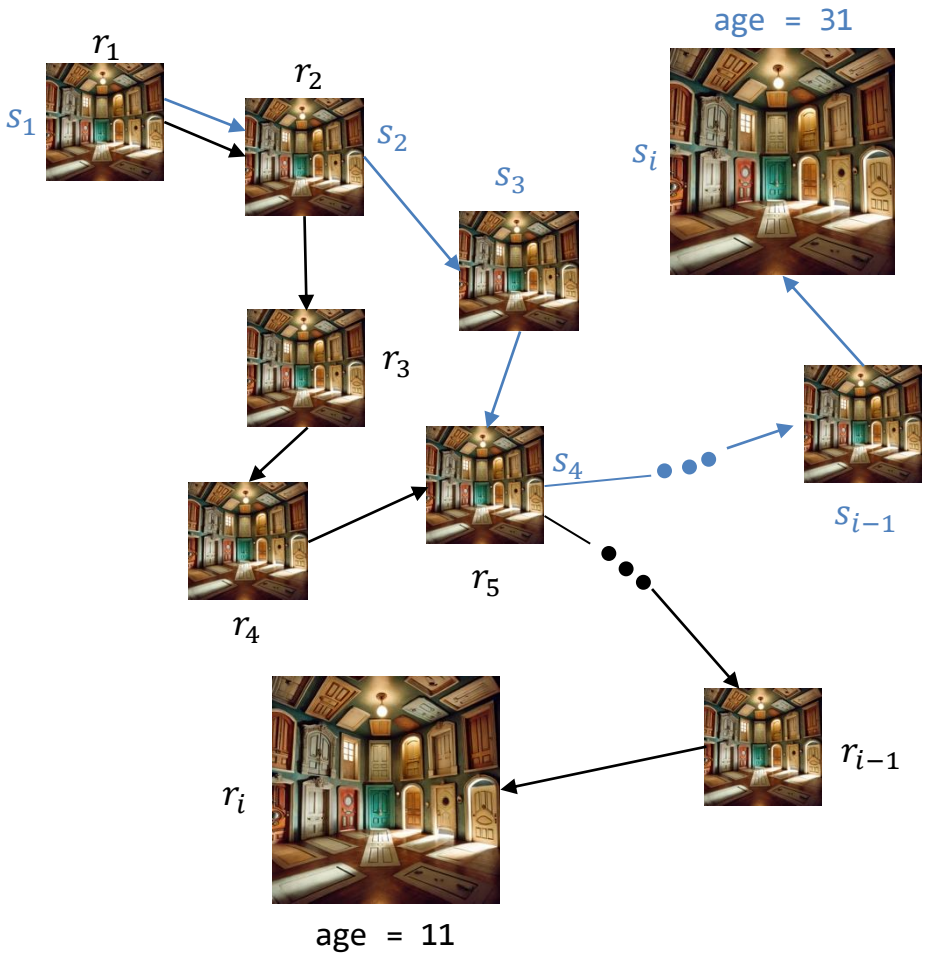


Teilprobleme identifizieren:

- Wir betrachten jetzt zwei Räume r_i und s_i und zwei Pfade $r_1, \ldots, r_i$ und $s_1, \ldots, s_i$ wobei r_1 und s_1 beide der Startraum room sind.

- Was müssen wir über die Teilsequenzen $r_1, \ldots, r_{i-1}$ und $s_1, \ldots, s_{i-1}$ wissen?

- Altersbedingung: An jeder Position i haben r_i und s_i dasselbe Alter: r_i.getAge() == s_i.getAge().



Teilprobleme identifizieren:

- Falls sich die Pfade bereits getrennt haben, so prüfen wir ob r_i und s_i beide Ausgänge sind und das gleiche Alter haben. (Falls ja dann sind wir fertig.)
- Sonst erkunden wir alle möglichen Pfadpaare (r_{i+1}, s_{i+1}).
- Wie merken wir uns ob sich die Pfade bereits getrennt haben?

Mit einem `boolean` Parameter

```java
// PRE: Beide Pfade gleich lang mit gleicher Altersfolge; samePath = bisher identischer Pfad
public static boolean solve2(Room room1, Room room2, boolean samePath) {

    if (room1.isExit() && room2.isExit() && !samePath) { // Zwei verschiedene Pfade enden am Ausgang
        return true;
    }
    for (int i = 0; i < room1.doorsTo.length; i++) { // Alle Raumpaare prüfen
        for (int k = 0; k < room2.doorsTo.length; k++) {
            if (room1.doorsTo[i].getAge() == room2.doorsTo[k].getAge()) { // Gleiches Alter?
                if (solve2(room1.doorsTo[i], room2.doorsTo[k],
                        samePath && room1.doorsTo[i] == room2.doorsTo[k])) { // Gleicher Pfad nur bei identischem Raum
                    return true; // Gültiges Pfadpaar gefunden
                }
            }
        }
    }
    return false; // Kein gültiges Pfadpaar gefunden
}
```

Wie nutzen wir diese Methode nun?

```java
public static boolean solve1(Room room, int sum) {
    // ...
}

public static boolean task1(Room room) {
    return solve1(room, 0); // Start mit Summe 0
}

public static boolean solve2(Room room1, Room room2, boolean samePath) {
    // ...
}

public static boolean task2(Room room) {
    return solve2(room, room, true); // Beide Pfade starten identisch im selben Raum
}
```

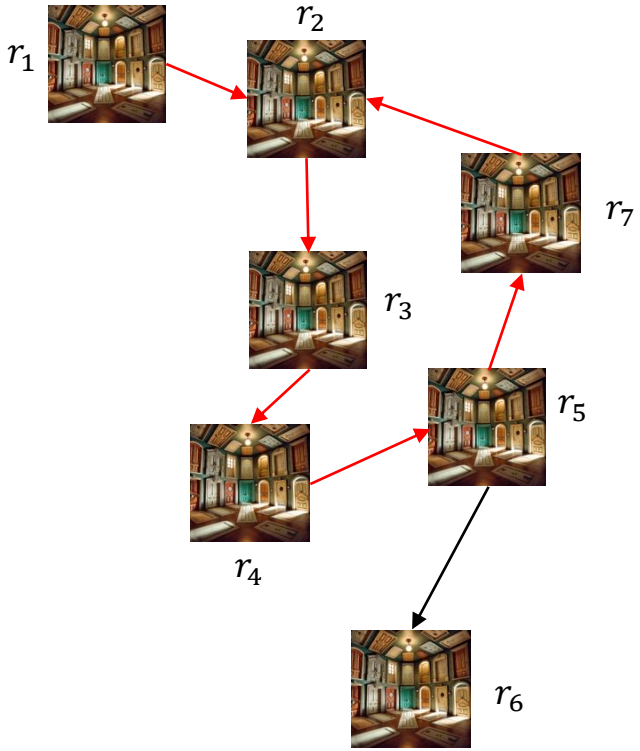
Mit gerichteten Zyklen arbeiten

Probleme Lösen: Labyrinth (modified)

Aufgabe 1: Labyrinth (2021 W8)

Ein Labyrinth besteht aus einer Menge von Räumen, welche durch die Klasse `Room` dargestellt werden. Die Klasse hat zwei Attribute: Der Integer `age` (grösser gleich 0) beschreibt das Alter des Raums und der Array `doorsTo` (nie `null`) beschreibt die Türen von diesem Raum zu anderen Räumen. Alle Türen sind Falltüren, d.h. sie funktionieren nur in eine Richtung. Ein Raum ist ein Ausgang aus dem Labyrinth, wenn keine Türen von dem Raum wegführen, das heisst, wenn `doorsTo` eine Länge von 0 hat.

Für alle Aufgaben werden Sie in einen zufälligen Raum geworfen, welcher als Argument gegeben wird (garantiert nicht `null`) und von welchem aus Sie die Aufgabe lösen müssen. ~~Sie dürfen für alle Aufgaben annehmen, dass es im Labyrinth keinen Zyklus gibt. Das heisst, dass man einen Raum, welchen man durch eine Türe verlassen hat, nie wieder erreichen kann indem man weiteren Türen folgt.~~ Eine Sequenz von N Räumen $r_1, \ldots, r_N$ ist ein *Lösungspfad* für einen Raum `room` genau dann wenn: (1) Der erste Raum r_1 ist der Raum `room`, (2) der letzte Raum r_N ist ein Ausgang, und (3) jeder Raum r_i mit $1 \leq i < N$ hat eine Tür zum nächsten Raum in der Sequenz r_{i+1}.



Gerichtete Zyklen führen zu Problemen

- Ohne Zyklenerkennung kann die Rekursion endlos weiterlaufen. Was soll visited speichern: jemals besucht oder auf dem aktuellen Pfad?
- Für diese Erweiterung suchen wir einfache Pfade: kein Raum zweimal.
- **Später:** Sets
- **Jetzt:** Benutzen eines `visited` Attributs.

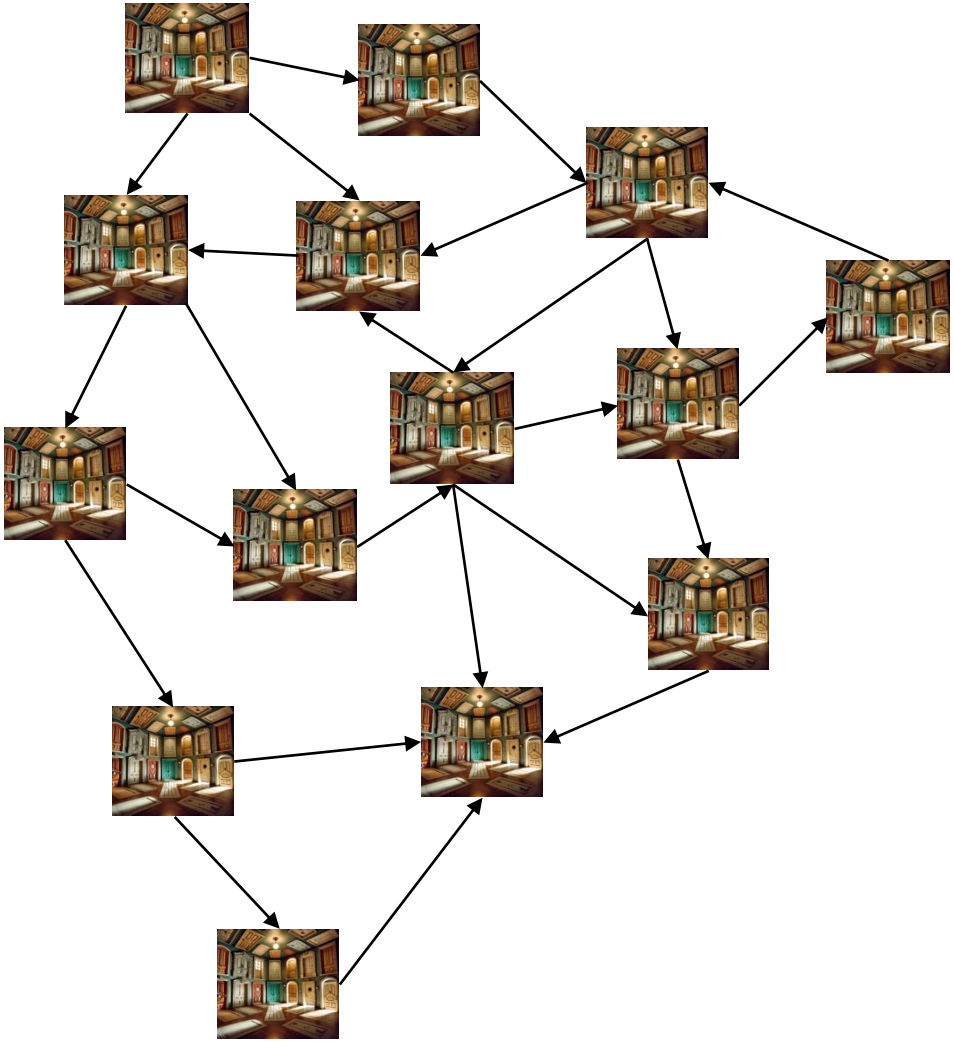
Probleme Lösen: Labyrinth

1. Implementieren Sie die Methode `Labyrinth.task1(Room room)`. Die Methode soll true zurückgeben genau dann, wenn es einen Lösungspfad $r_1, \ldots, r_N$ für `room` gibt, sodass:
   - Für jede Teilsequenz $r_1, ..., r_i$ mit $1 \leq i \leq N$ gilt, dass die Summe der Alter der Räume $r_1, ..., r_i$ nicht durch 3 teilbar ist.
2. Implementieren Sie die Methode `Labyrinth.task2(Room room)`. Die Methode soll true zurückgeben genau dann, wenn es zwei Lösungspfade $r_1, \ldots, r_N$ und $s_1, \ldots, s_N$ für `room` gibt, sodass:
   - Die Räume r_i und s_i haben das gleiche Alter für jedes i mit $1 \leq i \leq N$.
   - Für mindestens ein i mit $1 \leq i \leq N$ gilt, dass r_i und s_i unterschiedlich sind (verschiedene Referenzen).

Das können wir ausnutzen!

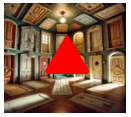
Sie dürfen Methoden und Felder der Klasse `Room` hinzufügen. Tests finden Sie in der Datei "LabyrinthTest.java".

Tipp: Lösen Sie die Aufgaben rekursiv. Für keine der Aufgaben müssen Sie alle Pfade generieren und dann erst prüfen, dass die Eigenschaften gelten. Manche der Tests enthalten Labyrinthe mit einer extrem grossen Anzahl an Pfaden aber leichten Lösungen.

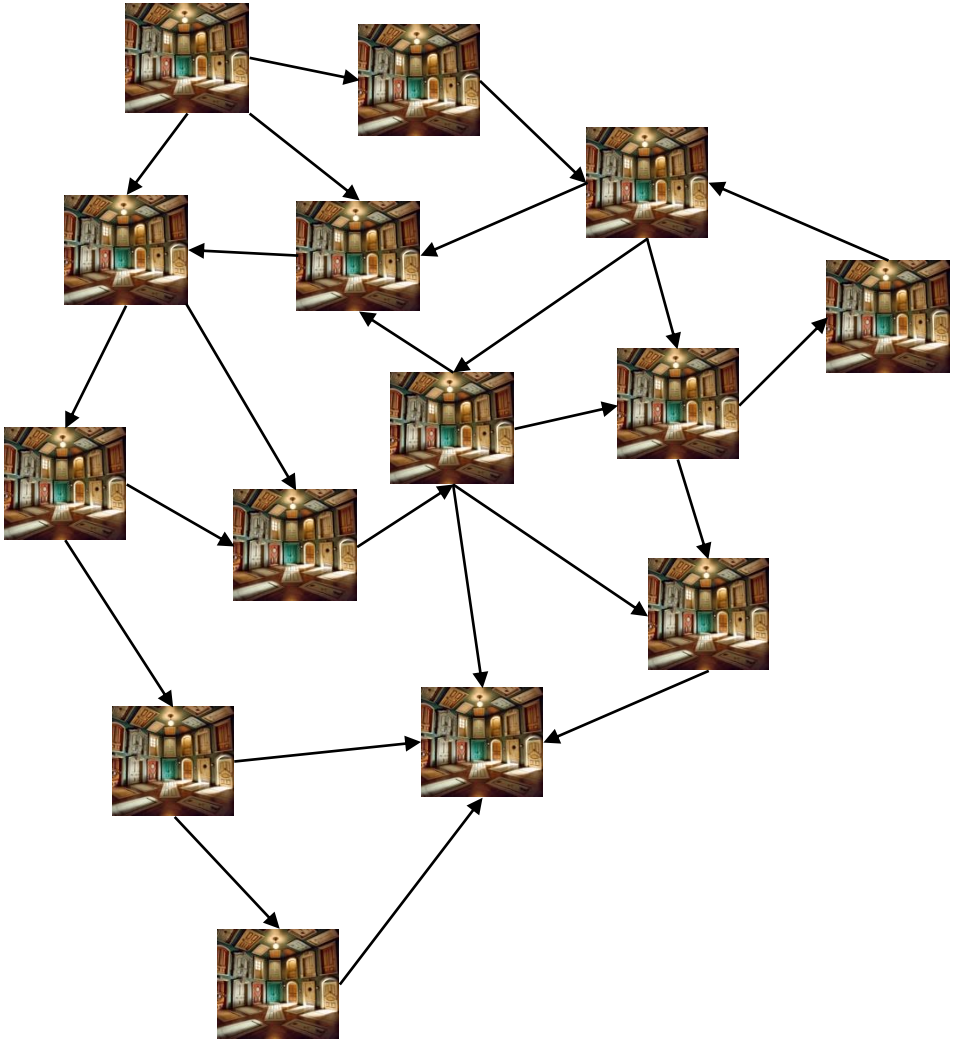


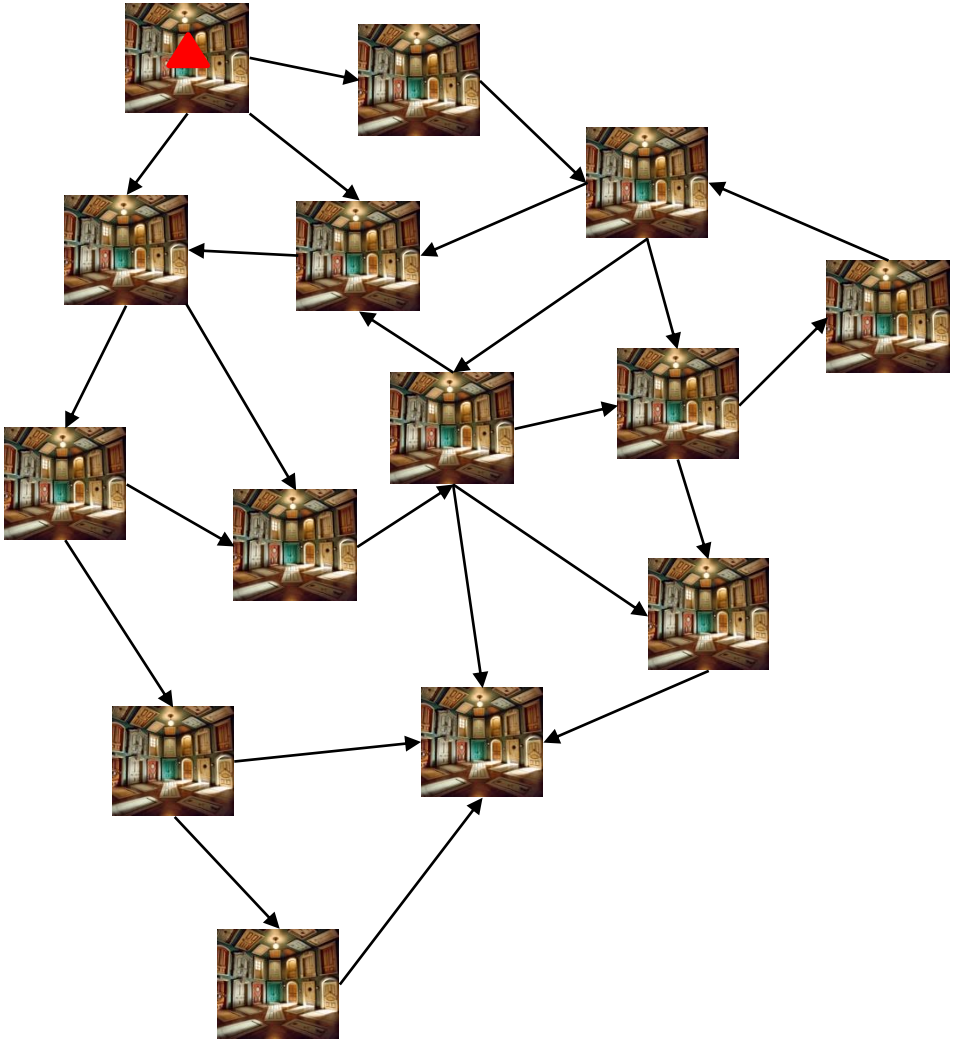


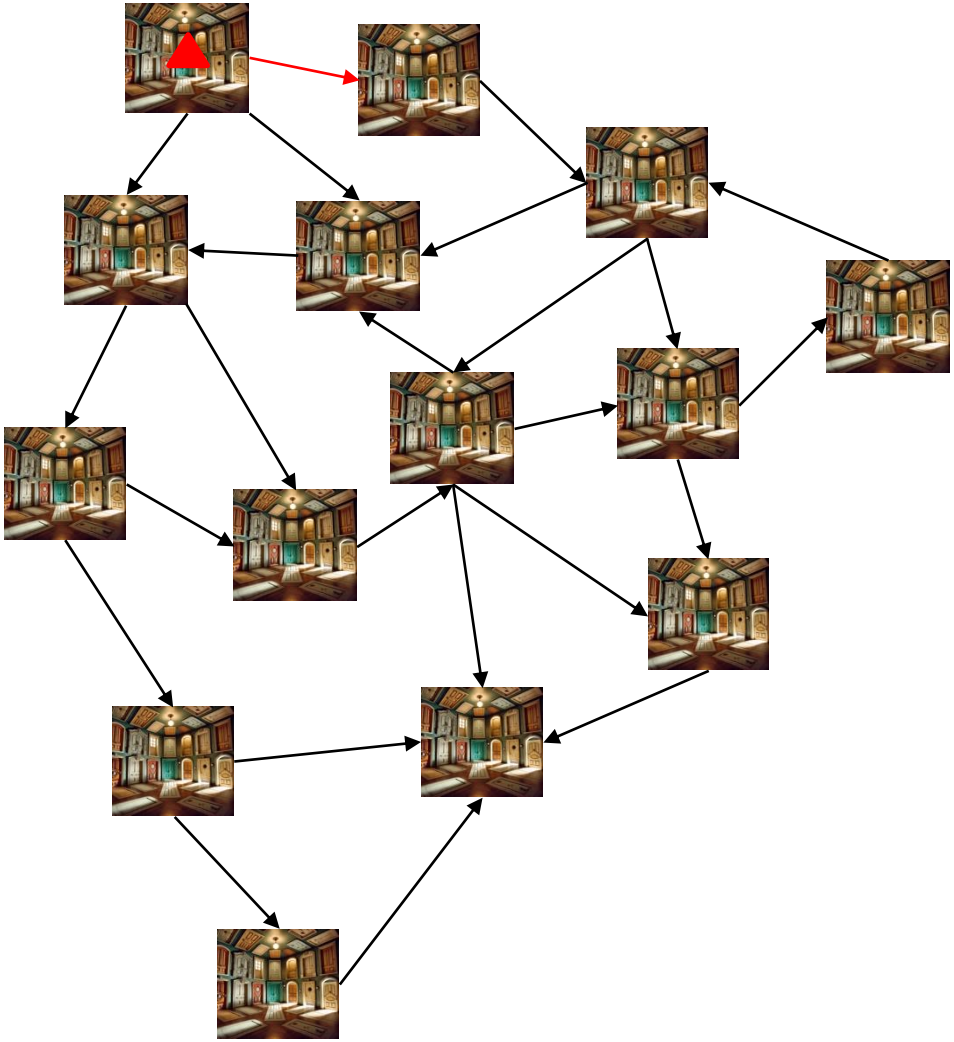
nicht visited

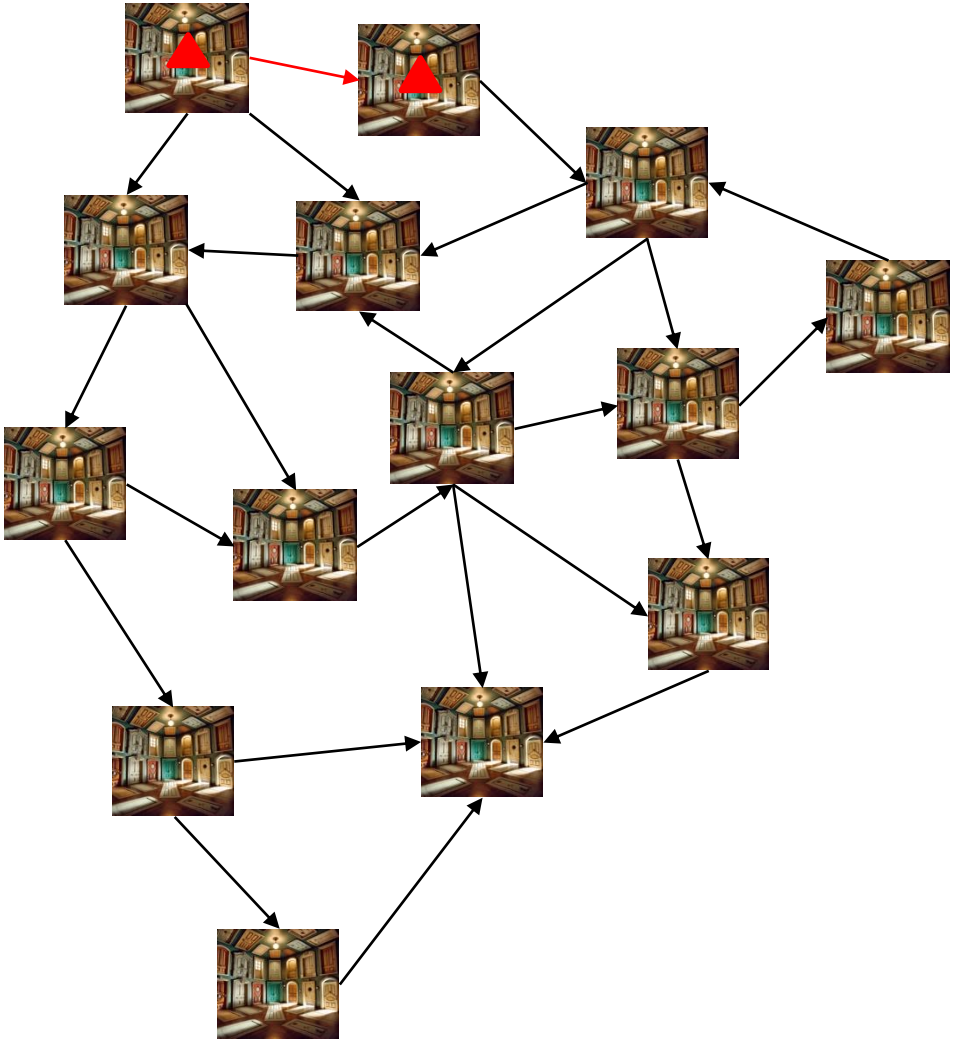


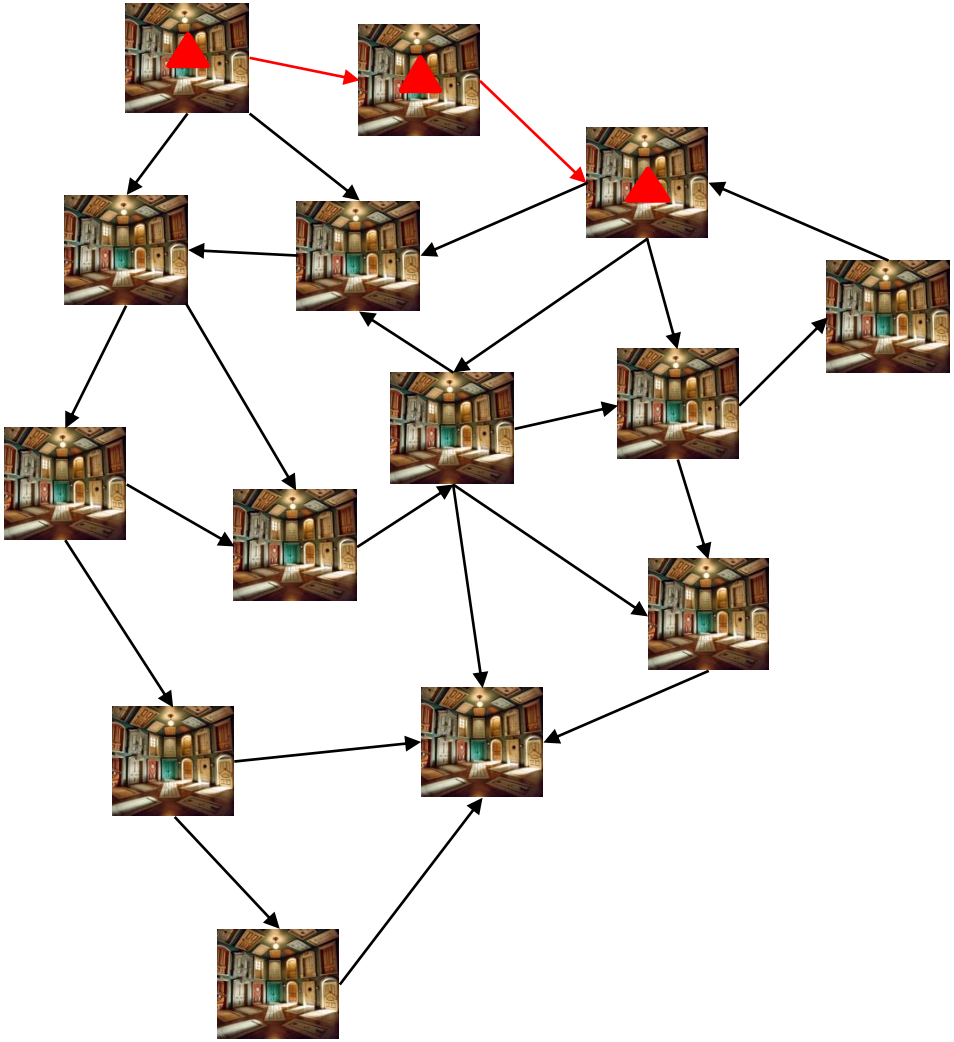
visited

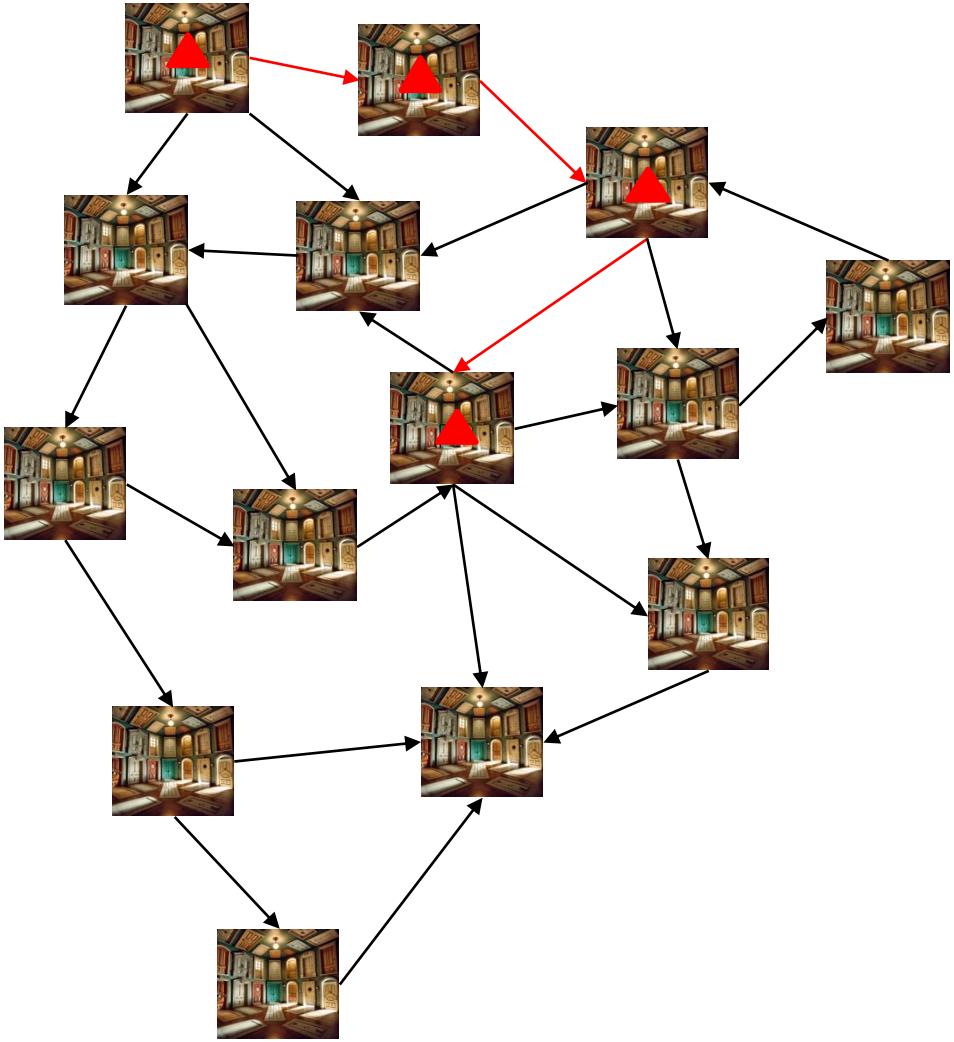


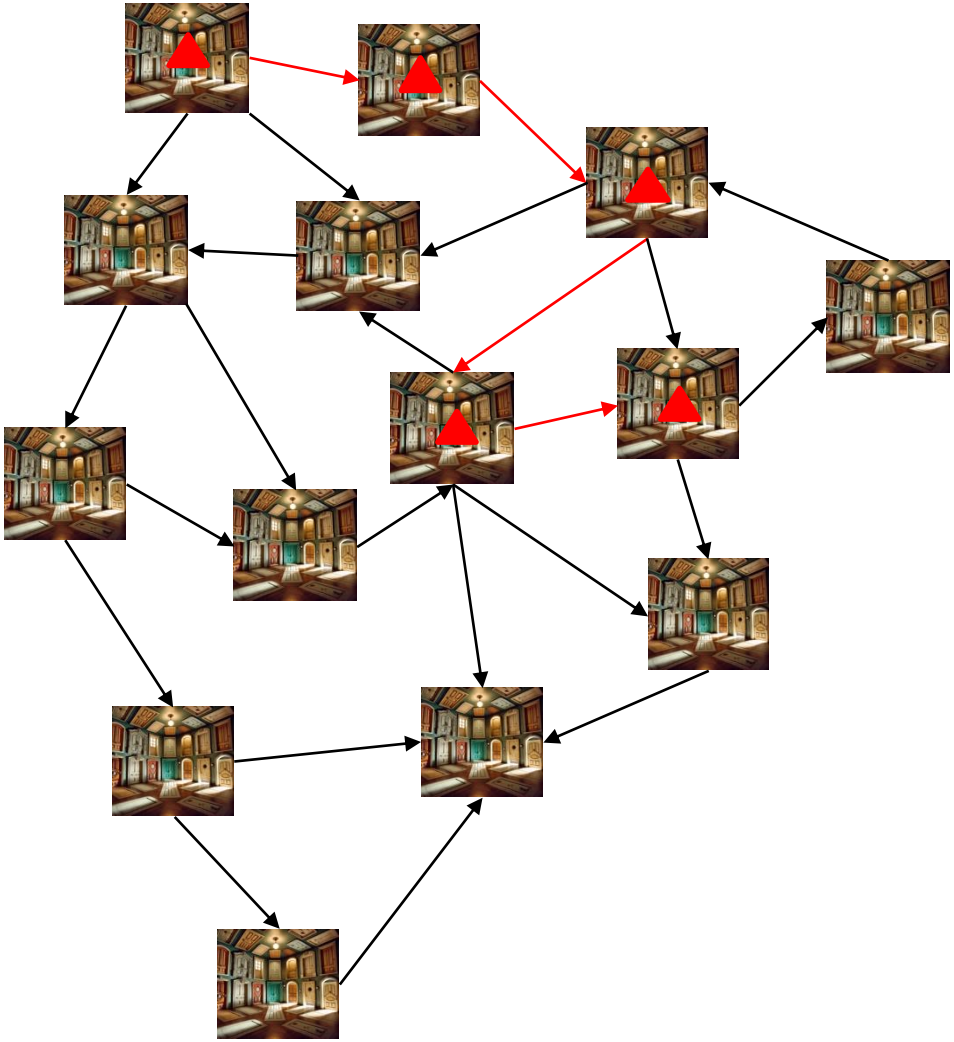


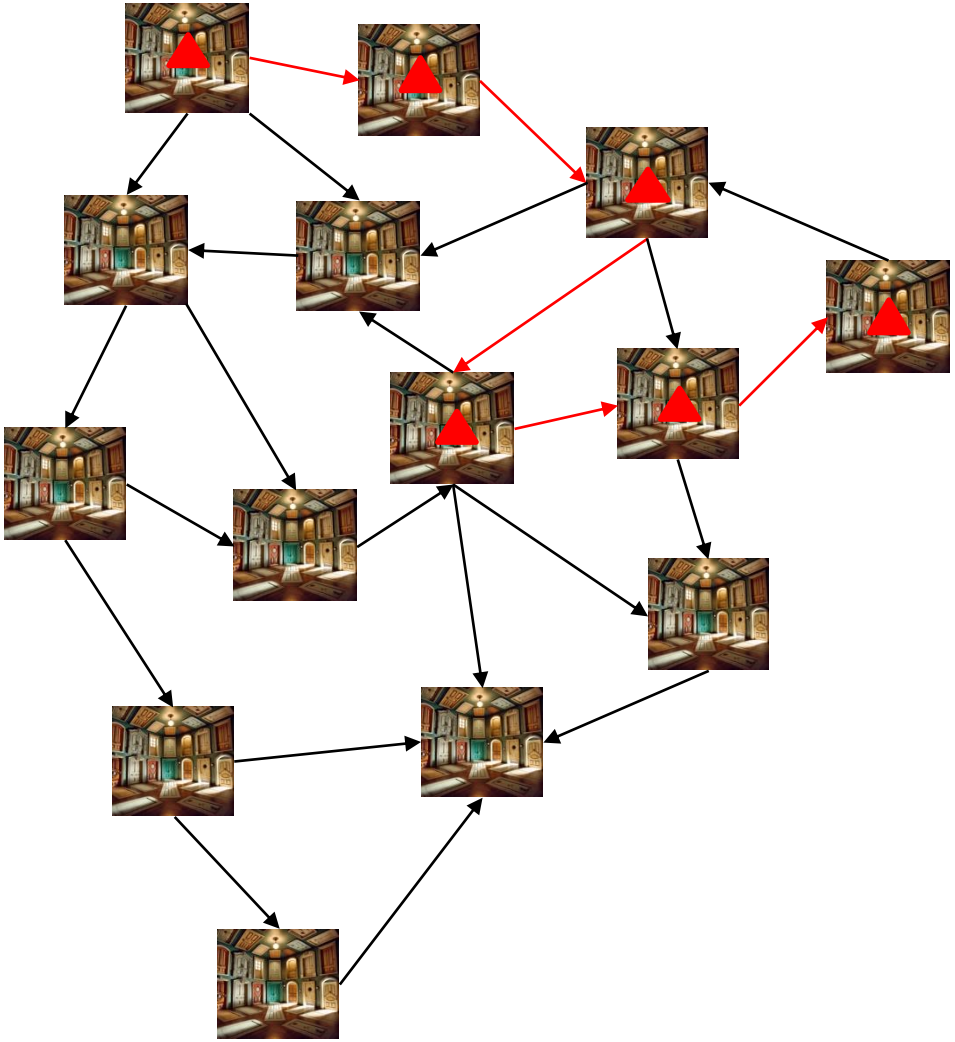




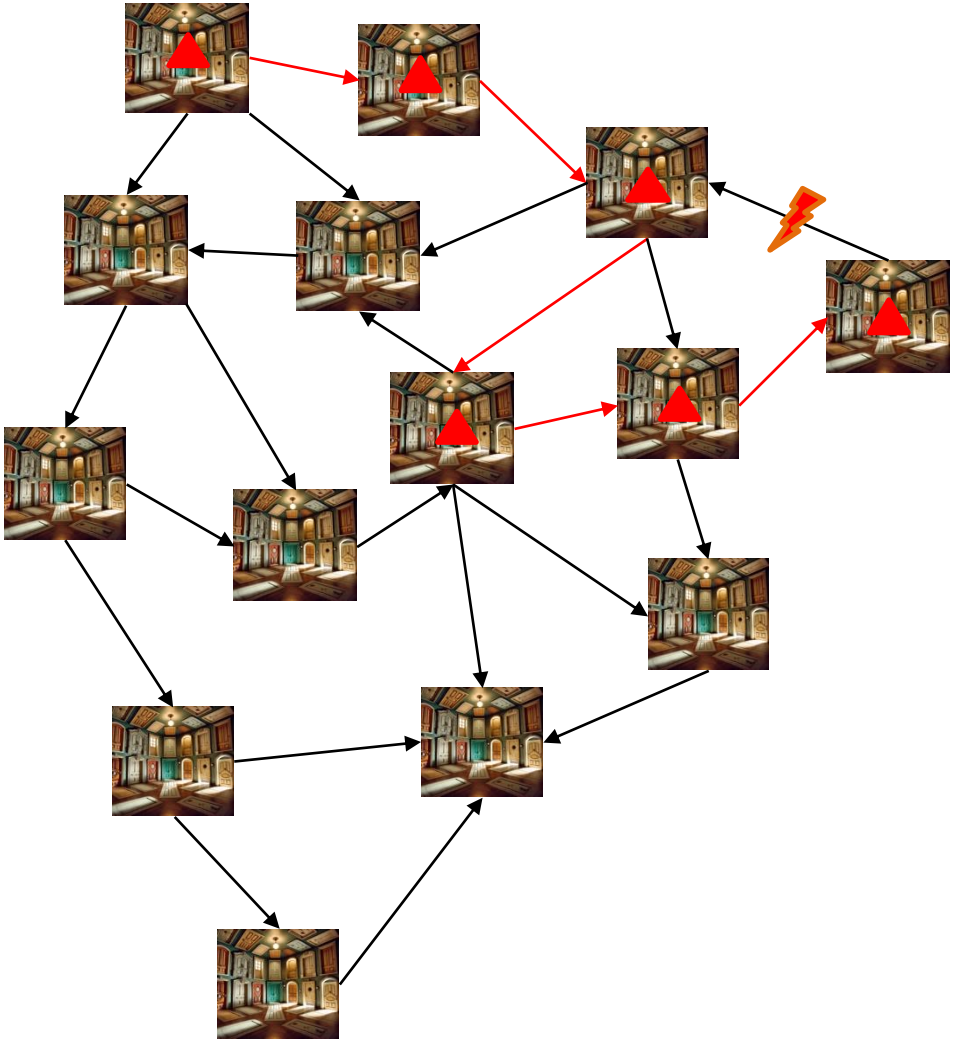




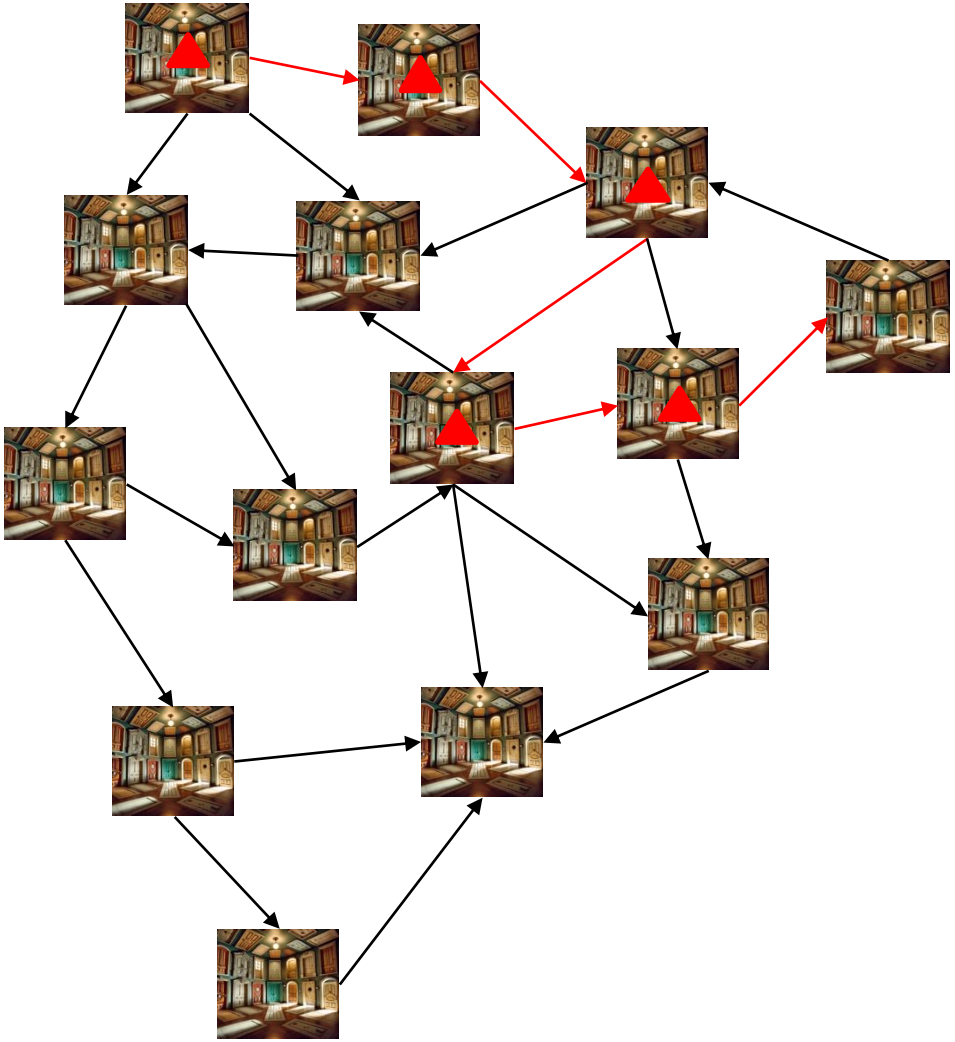




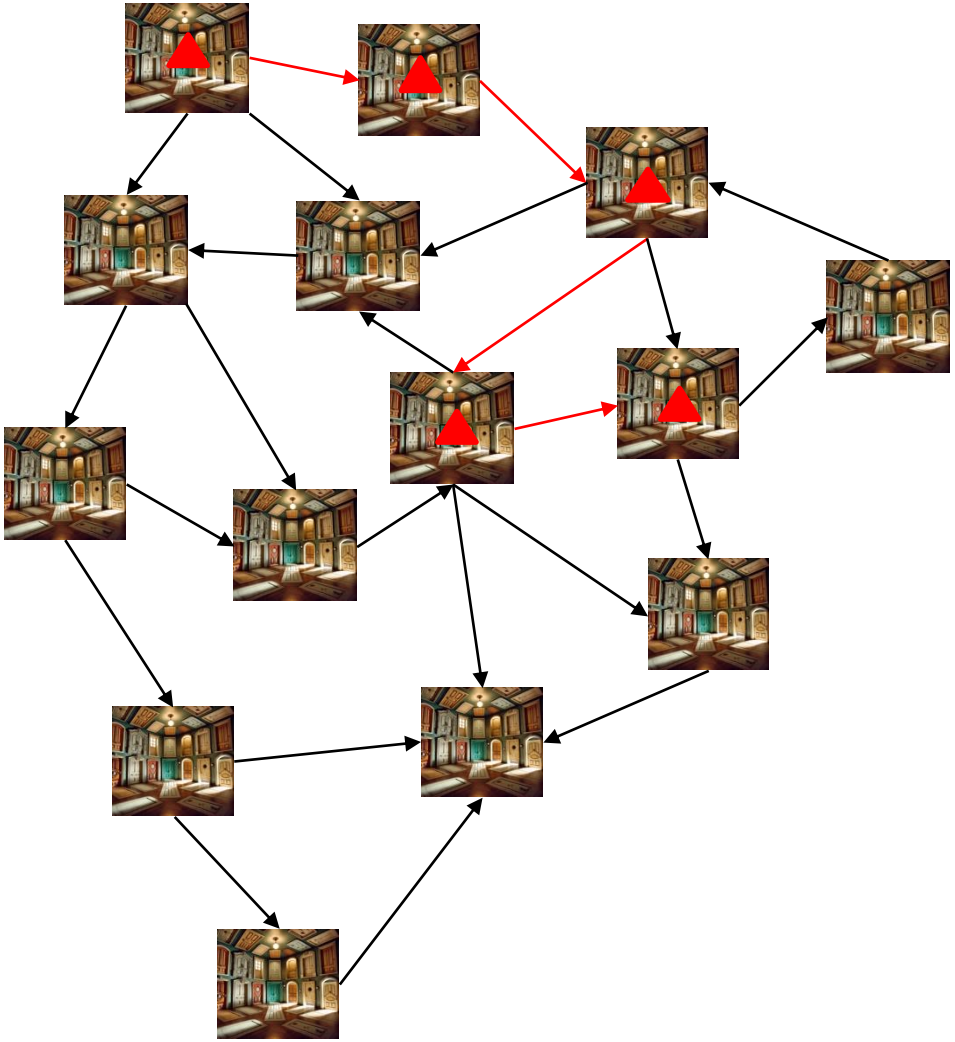
Was bewirkt visited hier?



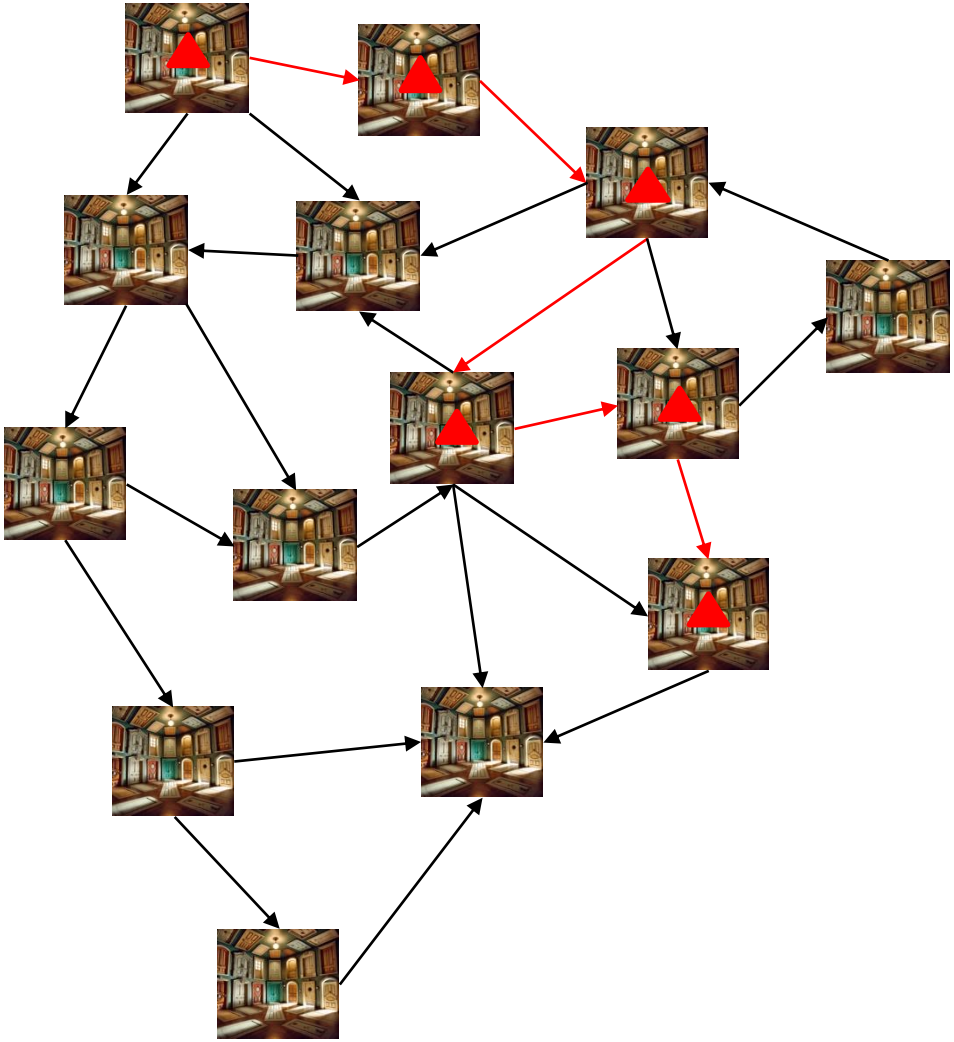
Was bewirkt visited hier?

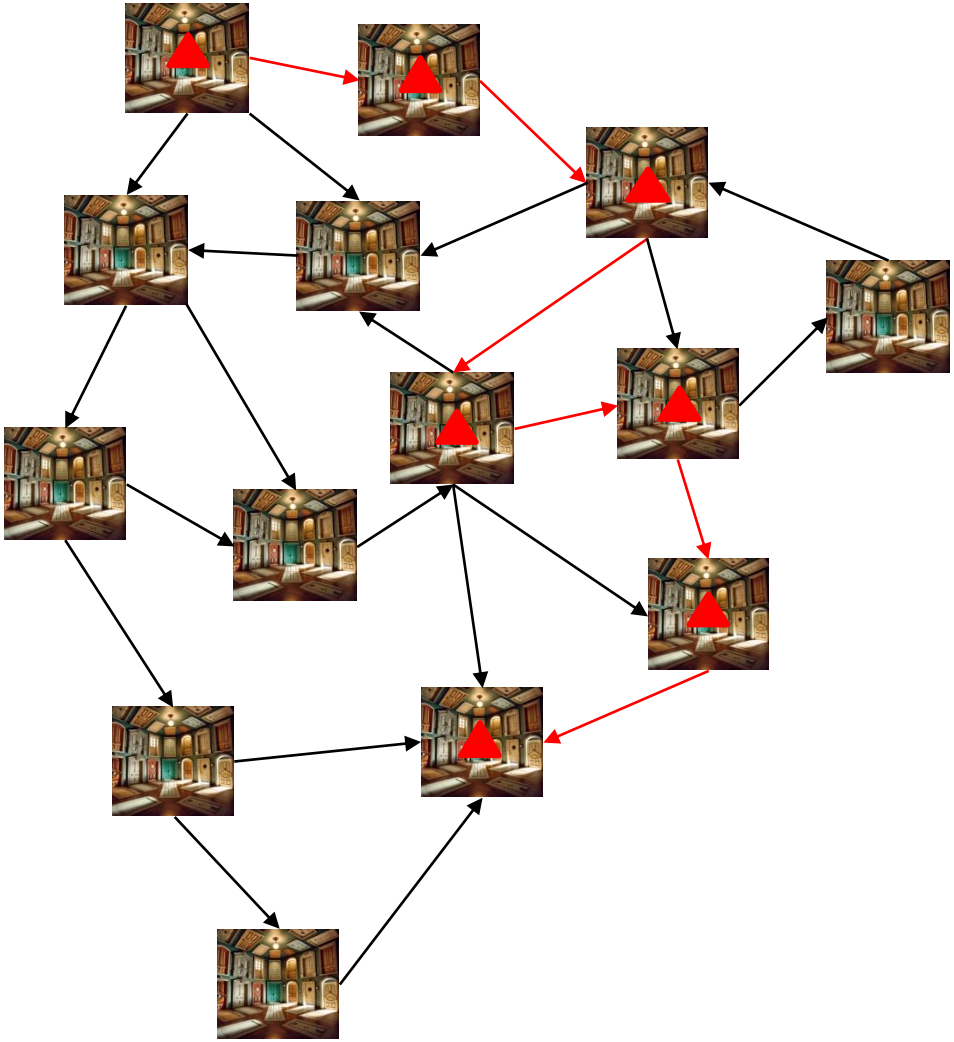


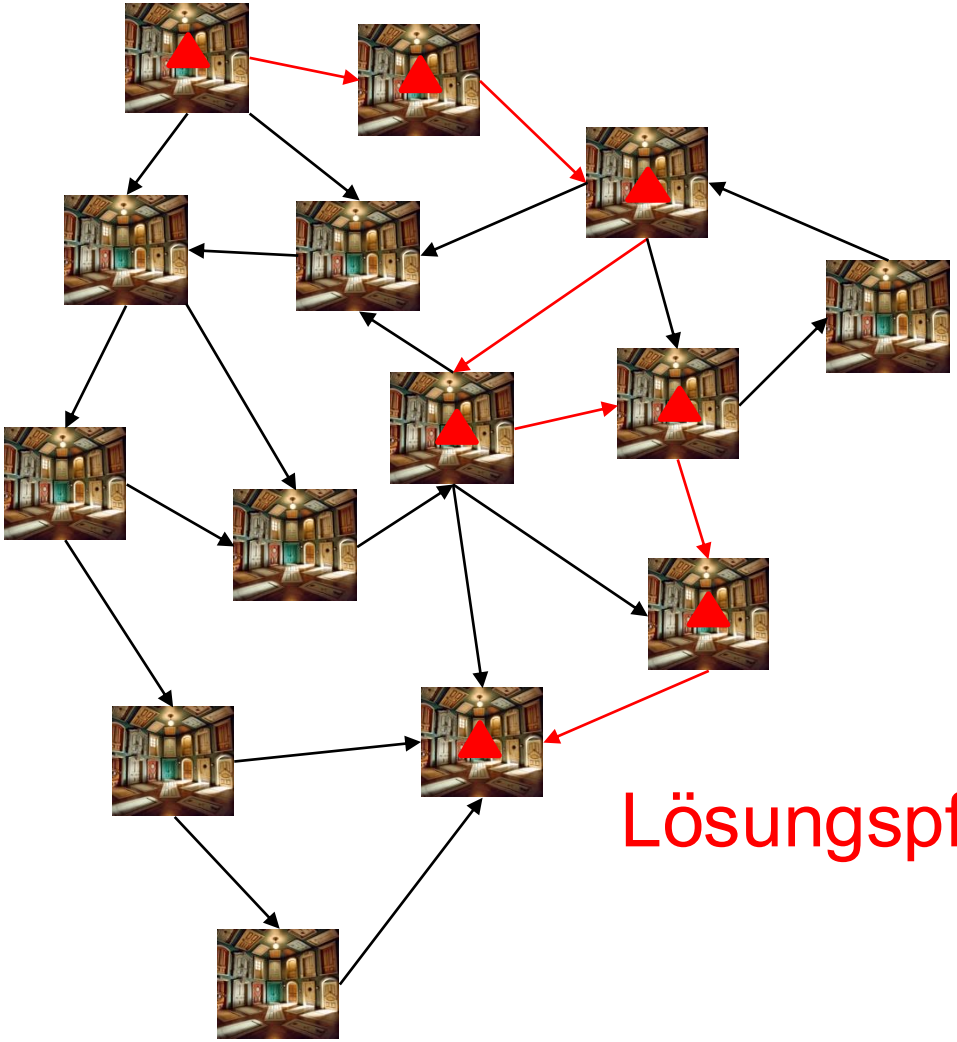
Was bewirkt visited hier?



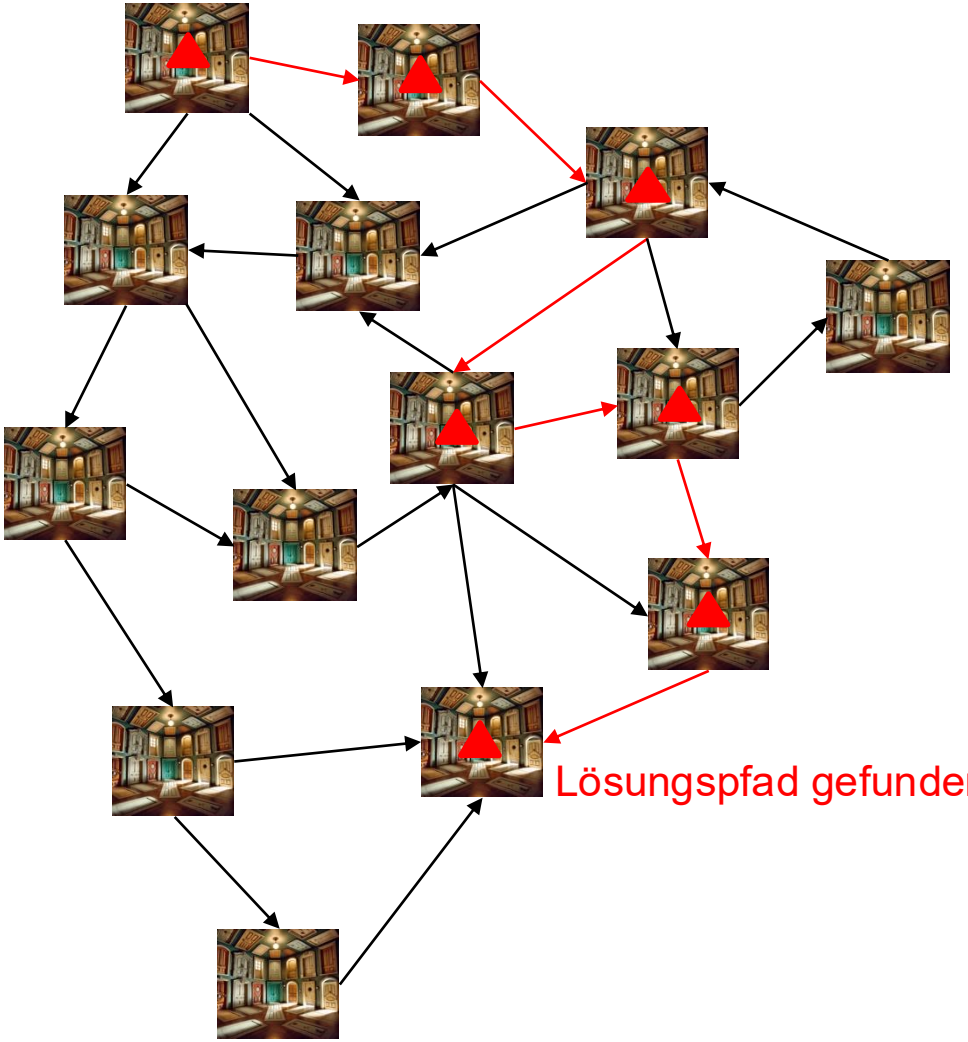
Kann dieser Room nochmal besucht werden in diesem Durchlauf?





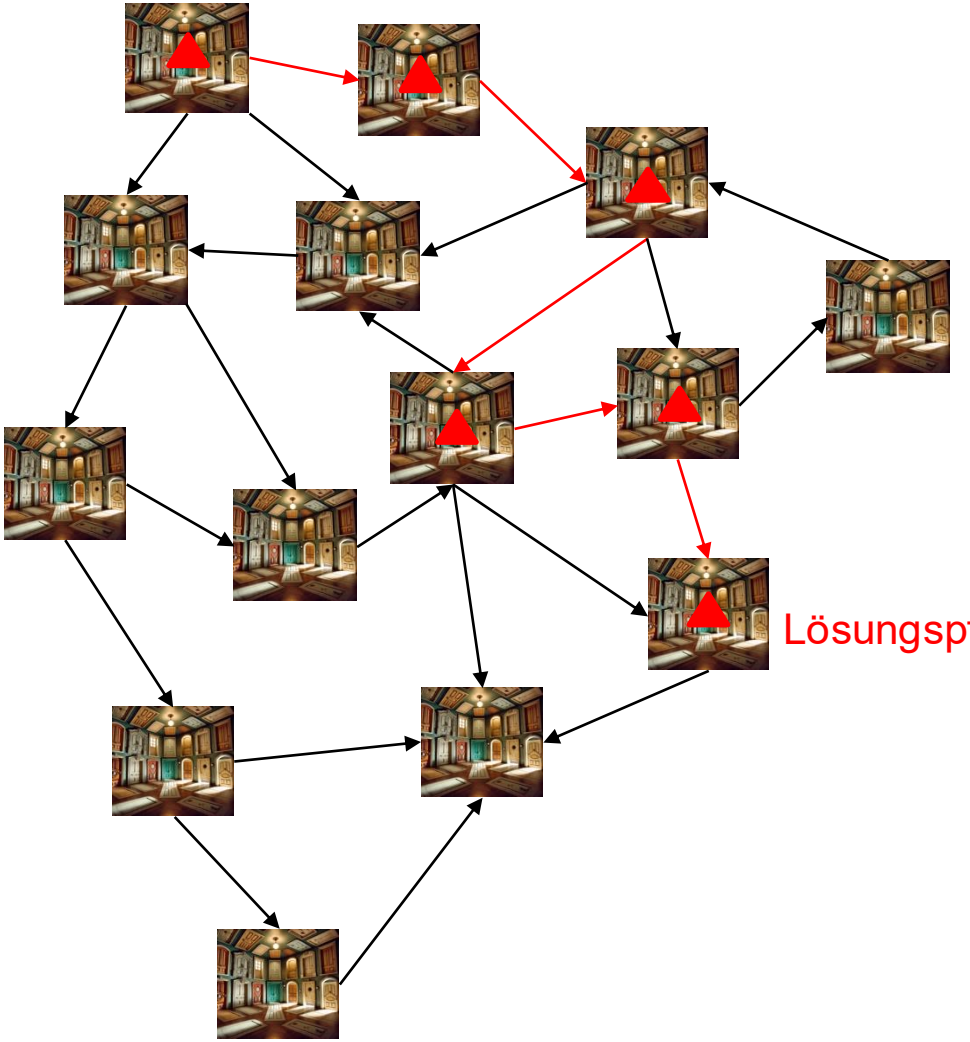


Lösungspfad gefunden!

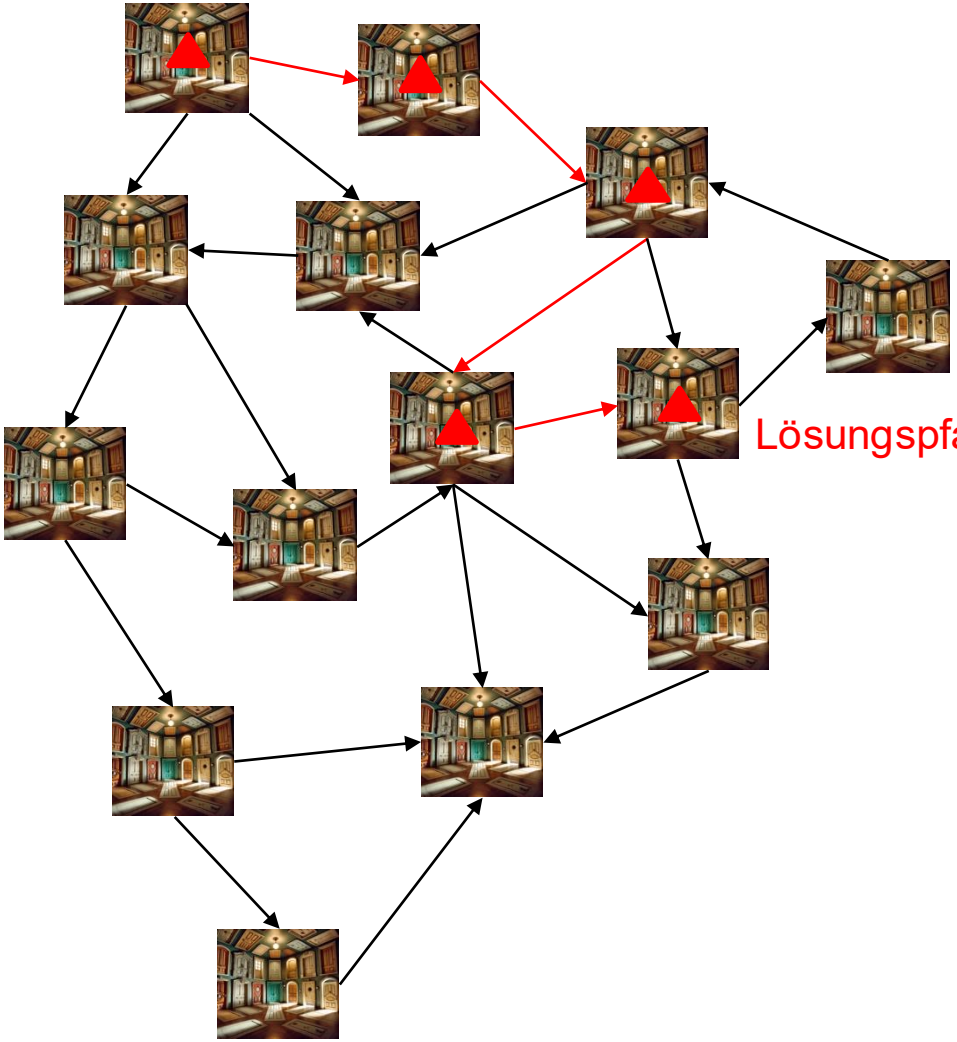


Lösungspfad gefunden!

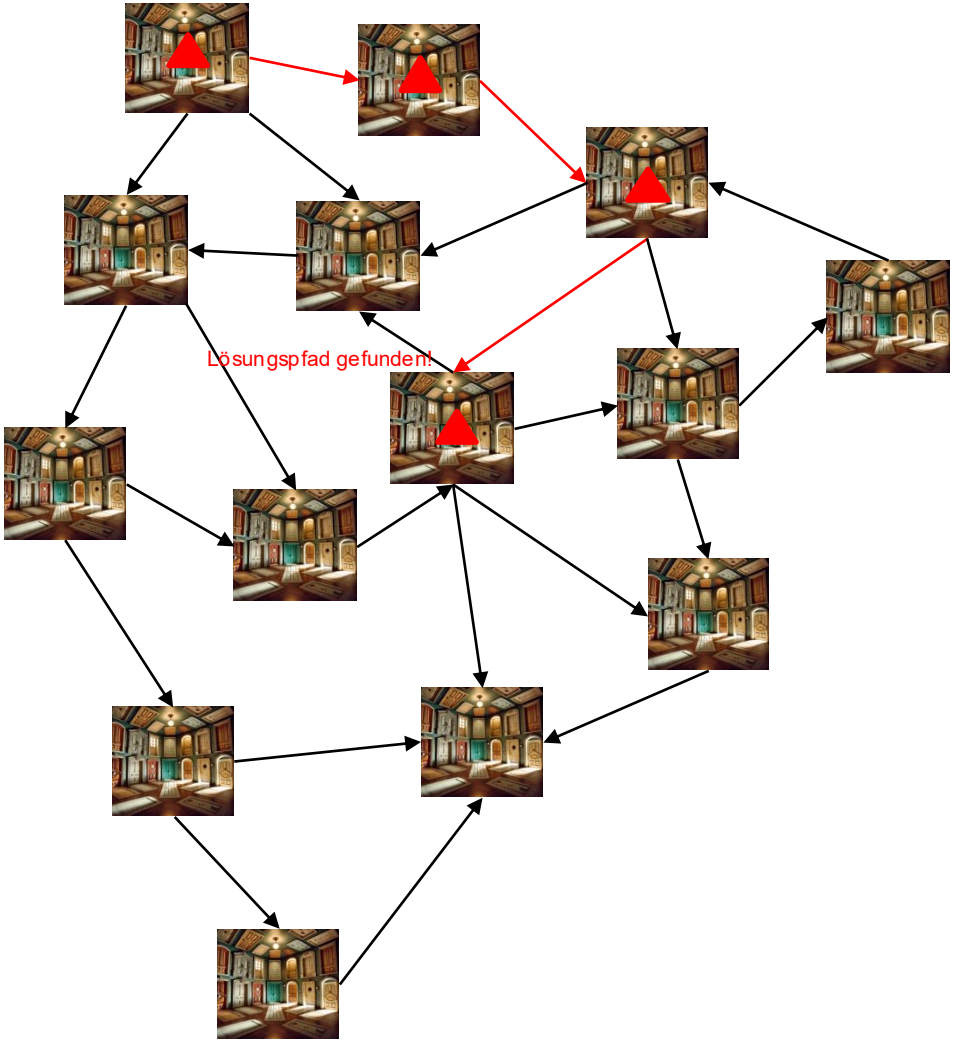
Was machen wir jetzt?

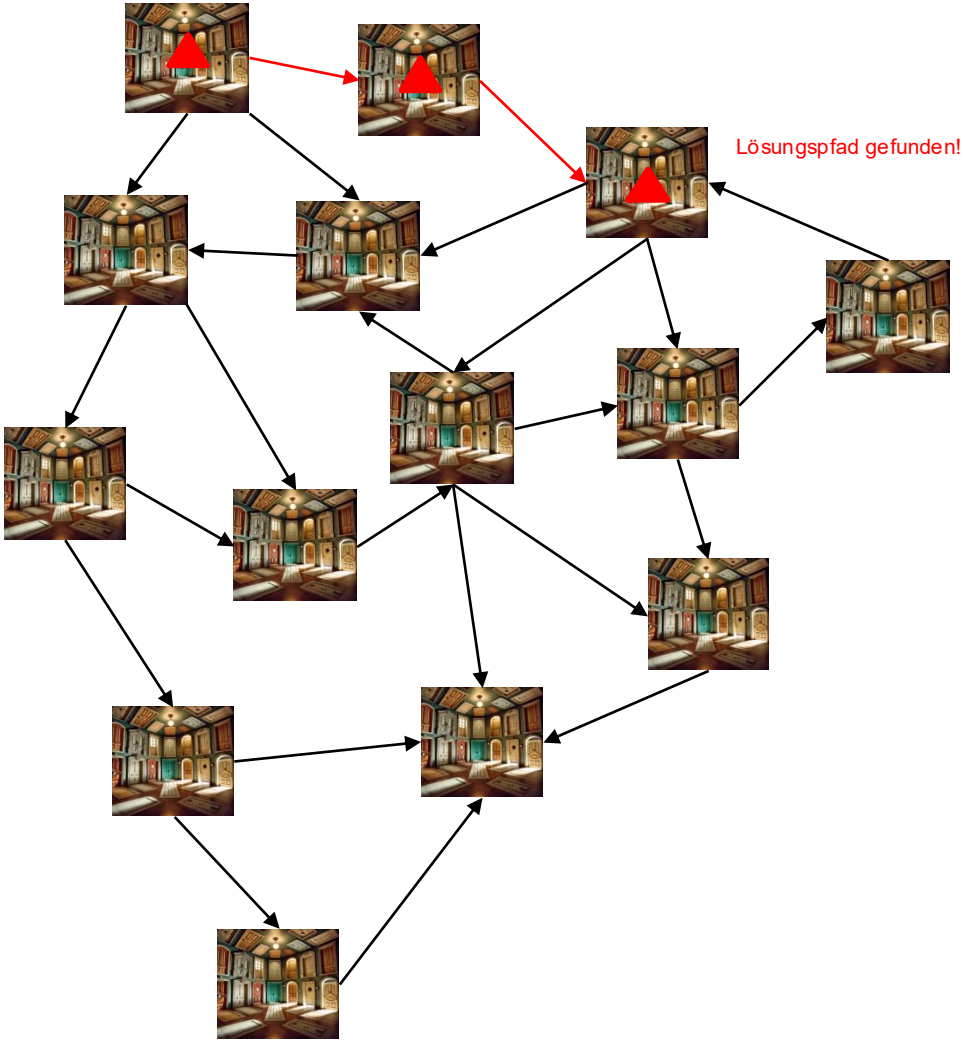


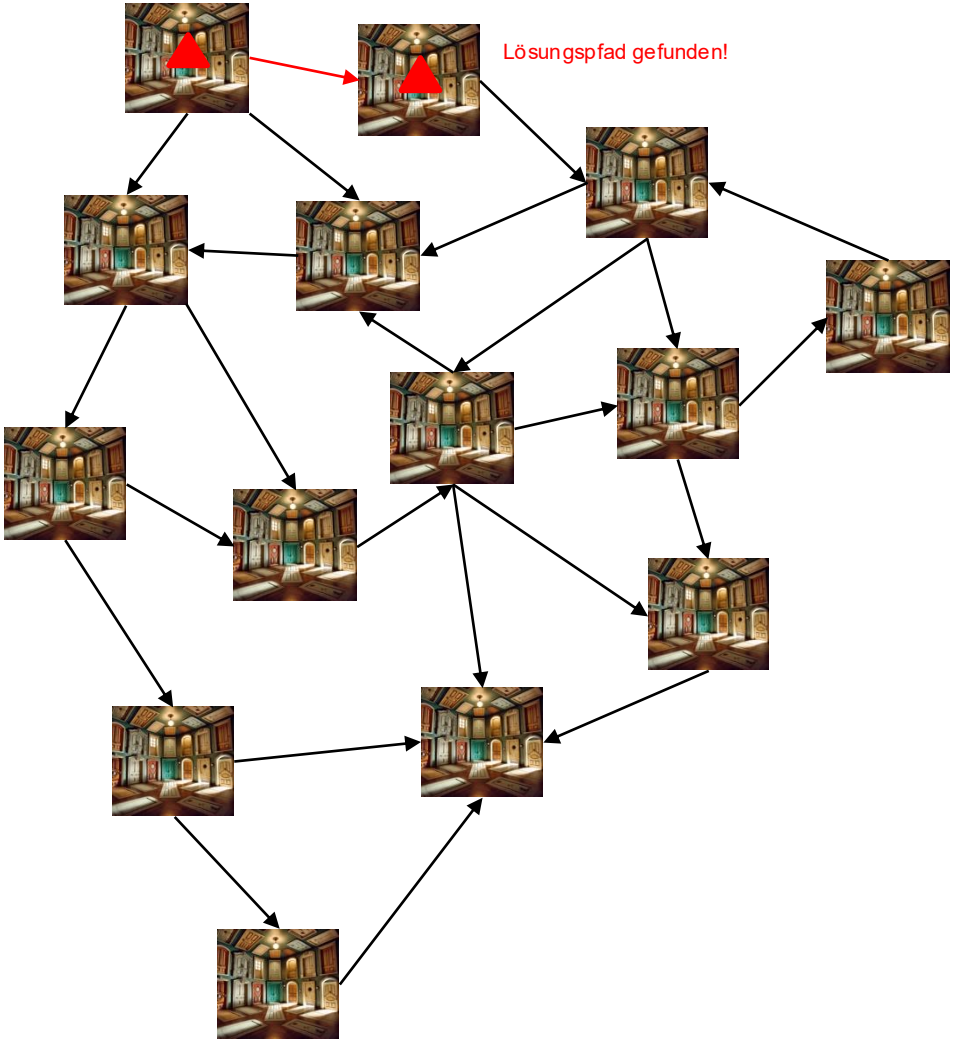
Lösungspfad gefunden!

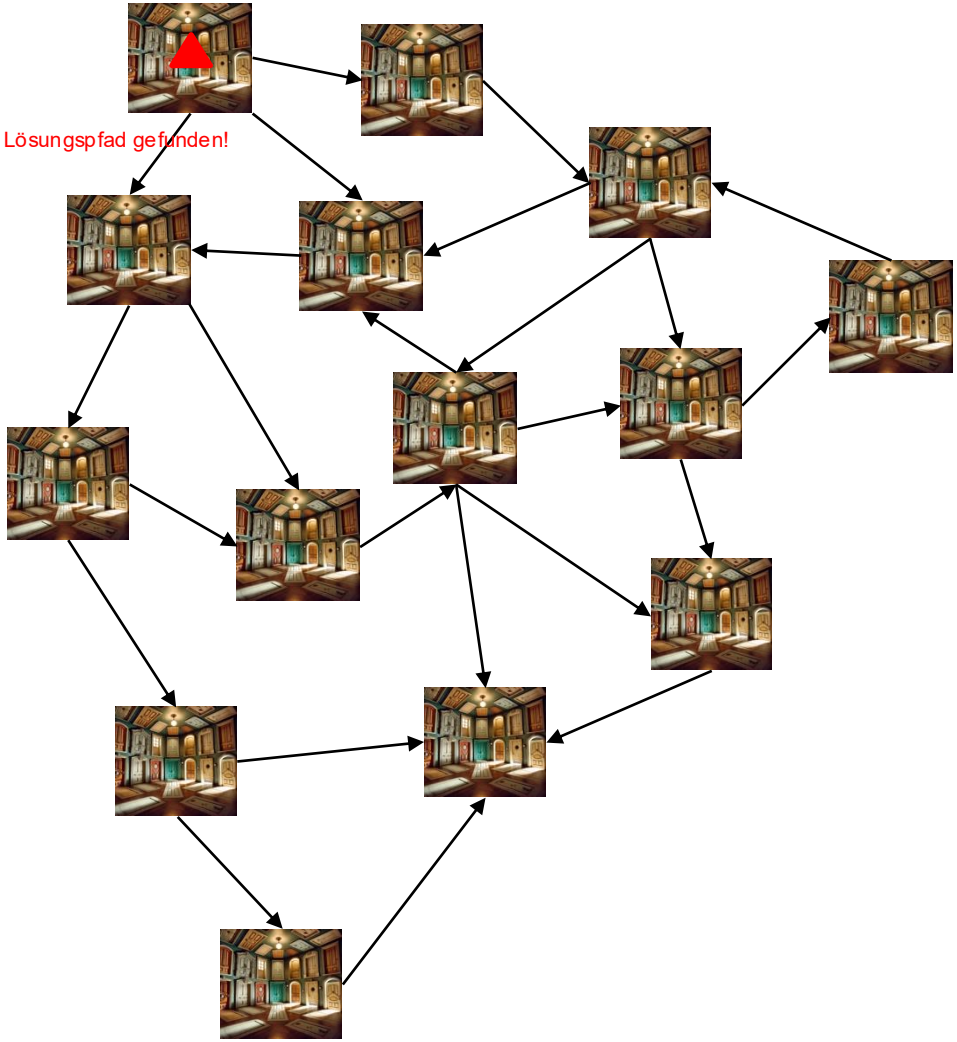


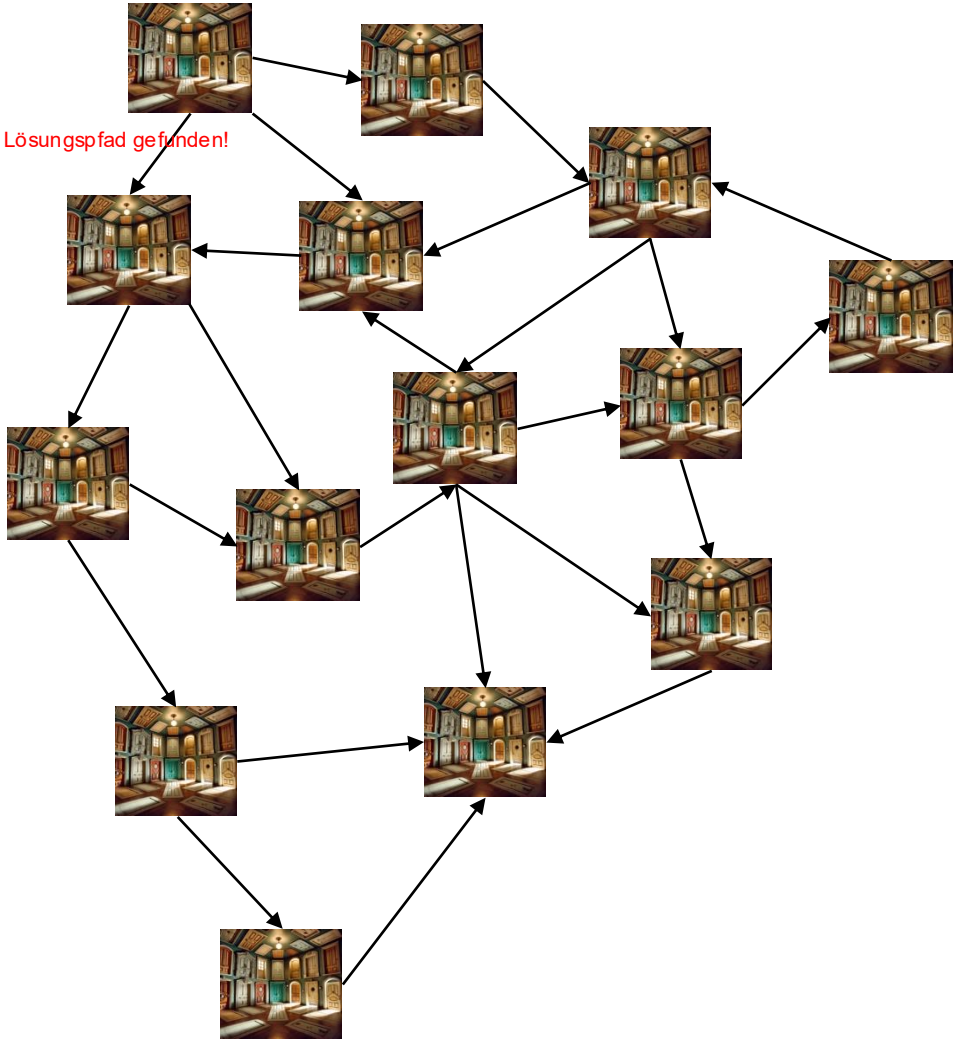
Lösungspfad gefunden!


Lösungspfad gefunden!


Lösungspfad gefunden!


Lösungspfad gefunden!


Lösungspfad gefunden!


Lösungspfad gefunden!

```java
public class Room {

    boolean visited = false; // Nur bei Zyklen: Auf aktuellem Pfad? Beim Zurückkehren resetten.

    private int age;          // Alter des Raums, >= 0
    public Room[] doorsTo;    // Türen zu anderen Räumen, nie null

    public Room(int age, Room... doorsTo) { // PRE: age >= 0; doorsTo und Elemente != null
        this.age = age;
        this.doorsTo = doorsTo;
    }

    public boolean isExit() { // Ausgang, wenn keine Türen
        return doorsTo.length == 0;
    }

    public int getAge() { // Alter zurückgeben
        return age;
    }
}
```

Initialisiert das `visited` Attribut für jedes Objekt mit `false`.

```java
public static boolean solve1(Room room, int sum) {
    if (!room.visited) {
        room.visited = true; // Kein Zyklus, Raum als besucht markieren

        sum = sum + room.getAge();
        if (sum % 3 == 0) {
            room.visited = false; // Zurücksetzen, damit andere Suchzweige den Raum wieder prüfen können
            return false;
        }
        if (room.isExit()) {
            room.visited = false; // Ausgangszustand wiederherstellen
            return true;
        }

        for (int i = 0; i < room.doorsTo.length; i++) {
            if (solve1(room.doorsTo[i], sum)) {
                room.visited = false; // Ausgangszustand wiederherstellen
                return true;
            }
        }
        room.visited = false; // Zurücksetzen, damit andere Suchzweige den Raum wieder prüfen können
    }
    return false; // Raum bereits auf dem Pfad (Zyklus) oder kein Weg führt zum Ausgang
}
```

Enums

Kontext: Mögliches Spiel

- Player haben
  - Namen – beliebig
  - Rolle – «honest» oder «trickster»
  - Energielevel – steigt/fällt während des Spiels
    - Änderung hängt (auch) von Rolle ab
- Game verwaltet alle Spieler in Player[] players
- Frage: Wie Spieler-Rollen umsetzen?

```
1 public class Player {
2   private ??? name;
3   private ??? role;
4   private int energy;
5   …
  }
```

Erste Version Player und Game

```java
public class Player {
  private String name;
  private String role;
  private int energy;
  …
  public void energize(int value) {
    energy += value;
  }
}
```

Rolle als Strings
modelliert – gute Idee?

```java
public class Game {
    private Player[] players;
    private Random random;
    …
    public void energizeAll() {
        for (int i = 0; i < players.length; i++) {
            Player player = players[i];
            if (player.role.equals("HONEST")) {
                player.energize(1);
            } else if (player.role.equals("TRICKSTER")) {
                player.energize(random.nextInt(-4, 5));
            }
        }
    }
}
```

Besser: Mit Enums

```java
public enum Role {
  HONEST,
  TRICKSTER,
  SOCERER,
}
```

```java
public class Player {
  private String name;
  private Role role;
  private int energy;
  …
}
```

Vorteile?

```java
public class Game {
    private Player[] players;
    private Random random;
    …
    public void energizeAll() {
        for (int i = 0; i < players.length; i++) {
            Player player = players[i];
            if (player.role == Role.HONEST) {
                player.energize(1);
            } else if (player.role == Role.TRICKSTER) {
                player.energize(random.nextInt(-4, 5));
            }
        }
    }
}
```

Theorieprüfung FS24

Aufgabe 2 (5)

Was druckt die Methode `main` der folgenden Klasse `Beispiel`?

```
class Beispiel {
    public static void main(String[] args) {

        int[] a = new int[6];

        int j = 0;

        for (int i=a.length-1; i>0; i-- ) {
            a[i] = 1 +  Math.min(i,j);
            j++;
        }

       int sum = 0;

       for (int k : a) {
           sum += a[k];
       }

        System.out.println("sum␣=␣" + sum);    // sum = ____________________

    }
}
```

Aufgabe 2 (5)

Was druckt die Methode `main` der folgenden Klasse `Beispiel`?

```
class Beispiel {
    public static void main(String[] args) {

        int[] a = new int[6];

        int j = 0;

        for (int i=a.length-1; i>0; i-- ) {
            a[i] = 1 +  Math.min(i,j);
            j++;
        }

        int sum = 0;

        for (int k : a) {
            sum += a[k];
        }

        System.out.println("sum = " + sum);     // sum = 14

    }
}
```

i	j	a
-	0	[0, 0, 0, 0, 0, 0]
5	1	[0, 0, 0, 0, 0, 1]
4	2	[0, 0, 0, 0, 2, 1]
3	3	[0, 0, 0, 3, 2, 1]
2	4	[0, 0, 3, 3, 2, 1]
1	5	[0, 2, 3, 3, 2, 1]

k	0	2	3	3	2	1
sum	0	0+3	0+3+3	0+3+3+3	0+3+3+3+3	0+3+3+3+3+2

Aufgabe 3 (4)

Gegeben sei in Abbildung 1 die EBNF-Beschreibung von *expression*. Für die EBNF-Beschreibung von *identifier* gelten die Regeln für Bezeichner in Java. Die EBNF-Beschreibung von *expression* unterscheidet sich aber sonst von den in Java zulässigen Ausdrücken.

$$
\begin{aligned}
\textit{shift_op} &\Leftarrow \boxed{\text{«}} \mid \boxed{\text{»}} \\
\textit{operator} &\Leftarrow \boxed{+} \mid \boxed{-} \mid \boxed{*} \mid \boxed{/} \\
\textit{factor} &\Leftarrow \textit{factor operator identifier} \mid \textit{identifier} \\
\textit{term1} &\Leftarrow \textit{factor} \boxed{*} \textit{identifier} \mid \textit{identifier} \\
\textit{term2} &\Leftarrow \boxed{(} \textit{factor operator identifier} \boxed{)} \mid \textit{identifier} \\
\textit{expression} &\Leftarrow \textit{term1 shift_op term2} \mid \textit{term2}
\end{aligned}
$$

Abbildung 1: **EBNF-Beschreibung** von *expression*

Geben Sie für jeden folgenden Ausdruck an, ob er nach der EBNF-Beschreibung von *expression* in Abbildung 1 gültig ist. (Tipp: alle Bezeichner und Werte in diesen Ausdrücken (d.h., *identifier* in der Abbildung 1) sind korrekt.)

Ausdruck	Gültig	Ungültig	Ausdruck	Gültig	Ungültig
`a « ( 5 + b )`	X		`b « ( x + y * b )`	X	
`v » ( u - u )`	X		`r « s * ( t + t )`		X
`k « ( l * ( m + n ) )`		X	`( a * b ) » c`		X

Aufgabe 6 (5)

Gegeben ist die Methode `compute(int n, int d)`, die für eine ganze Zahl $n \geq 1$ den Ausdruck

$$\frac{n \cdot (n-1) \cdot d}{2}$$

berechnet. (Sie können davon ausgehen, dass die Parameter eines Methodenaufrufs sicherstellen, dass alle Berechnungen korrekt und ohne Overflow oder Underflow ausgeführt werden.)

Was ist die Invariante für den Loop in dieser Methode?

```
    static int compute(int n, int d) {
 // Precondition: n >= 1
   int res = 0;
   int k = 1;
 // Loop Invariante:  Bitte geben Sie die Invariante unten an
   while (k <= n)   {
      res = res + (k-1) * d;
      k = k + 1;
   }
// Postcondition: res == n·(n−1)·d / 2
       return res;
}
```

Geben Sie die Loop Invariante an. (Sie können, wenn Sie es für nötig halten, die Precondition oben erweitern.)

Loop Invariante: k <= n+1 && res == d*(k-2)*(k-1)/2 && n >= 1 && k >= 1

_____

_____

1

k <= n+1

3

n >= 1
k >= 1

2

i	k	res
0	1	0
1	2	0
2	3	d
3	4	d+2d
4	5	d+2d+3d
5	6	d+2d+3d+4d
…	…	…
i	i+1	d*(i-1)*i/2

k == i+1 ⇔ i == k-1
=> res == d*k-2)*(k-1)/2

Aufgabe 7 (6)

Bitte geben Sie für die folgenden Java-Programmsegmente die schwächste Vorbedingung (weakest precondition) WP an. Bitte geben Sie die Precondition als Java-Expression an. Alle Variablen sind vom Typ `int` und initialisiert; es gibt keinen Over-/Underflow.

1.
```
WP: {                                                    }

    f = e * 2;
    g = f + 3;

    Q: { g >= 12 }
```

Aufgabe 7 (6)

Bitte geben Sie für die folgenden Java-Programmsegmente die schwächste Vorbedingung (weakest precondition) WP an. Bitte geben Sie die Precondition als Java-Expression an. Alle Variablen sind vom Typ `int` und initialisiert; es gibt keinen Over-/Underflow.

1. `WP: {` e * 2 >= 9 `}`

```
f = e * 2;
g = f + 3;

Q: { g >= 12 }
```

e * 2 + 3 >= 12

f + 3 >= 12

2. WP: {

```
y = x * x;
z = (x+1) * (x+1);
w = y + z;

Q: { w > 0 }
```

}

Vereinfacht: true

2. WP: { x*x + (x+1) * (x+1) > 0 }

```
y = x * x;
z = (x+1) * (x+1);
w = y + z;

Q: { w > 0 }
```

x*x + (x+1) * (x+1) > 0

y + (x+1) * (x+1) > 0

y + z > 0

3. WP: {

```
  r = s + 2;
  if (r < 0) {
    t  = s * s;
  } else {
    t = r - 4;
  }

  Q: { t > 0 }
```

}

Vereinfacht: s < -2 || s > 2

3. WP: { (s + 2 < 0 && s*s > 0) || (s + 2 >= 0 && s – 2 > 0) }

```
r = s + 2;          s*s > 0                    s + 2 - 4 > 0
if (r < 0) {
  t  = s * s;       s*s > 0
} else {
  t = r - 4;                                    r - 4 > 0
}

Q: { t > 0 }
```

Vorbesprechung

Ranking

- **Prüfungsrelevant**

- **Empfehlenswert**

- **Nicht prüfungsrelevant**

Aufgabe 1: Loop-Invariante

Gegeben ist eine Postcondition für das folgende Programm

```
public int compute(String s, char c) {
  // Precondition s != null
  int x;
  int n;

  x = 0;
  n = 0;

  // Loop Invariante:
  while (x < s.length()) {
    if (s.charAt(x) == c) {
      n = n + 1;
    }
    x = x + 1;
  }

  // Postcondition: count(s, c) == n
  return n;
}
```

Die Methode `count(String s, char c)` gibt zurück wie oft der Character `c` im String `s` vorkommt. Schreiben Sie die Loop Invariante in die Datei "LoopInvariante.txt". **Tipp:** Benutzen Sie die `substring` Methode.

Aufgabe 2: Linked List

Bisher haben Sie Arrays verwendet, wenn Sie mit einer grösseren Anzahl von Werten gearbeitet haben. Ein Nachteil von Arrays ist, dass die Grösse beim Erstellen des Arrays festgelegt werden muss und danach nicht mehr verändert werden kann. In dieser Aufgabe implementieren Sie selbst eine Datenstruktur, bei welcher die Grösse im Vornherein nicht bestimmt ist und welche jederzeit wachsen und schrumpfen kann: eine *linked list* oder *verkettete Liste*.

Eine verkettete Liste besteht aus mehreren Objekten, welche Referenzen zueinander haben. Für diese Aufgabe besteht jede Liste aus einem "Listen-Objekt" der Klasse `LinkedIntList`, welches die gesamte Liste repräsentiert, und aus mehreren "Knoten-Objekten" der Klasse `IntNode`, eines für jeden Wert in der Liste. Die Liste heisst "verkettet", weil jedes Knoten-Objekt ein Feld mit einer Referenz zum nächsten Knoten in der Liste enthält. Das `LinkedIntList`-Objekt schliesslich enthält eine Referenz zum ersten und zum letzten Knoten und hat ausserdem ein Feld für die Länge der Liste.

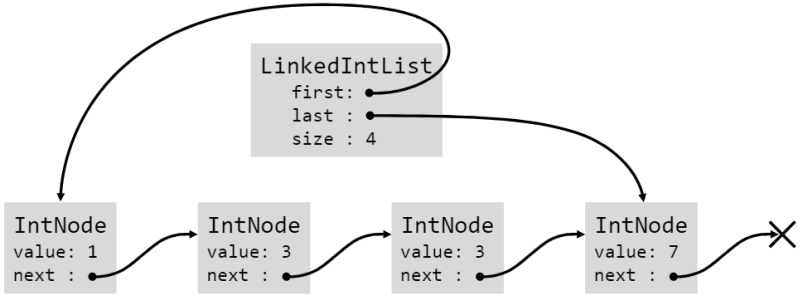


Abbildung 1: Verkettete Liste mit Werten 1, 3, 3, 7.

Aufgabe 2: Linked List

Name	Parameter	Rückg.-Typ	Beschreibung
`addLast`	`int value`	`void`	fügt einen Wert am Ende der Liste ein
`addFirst`	`int value`	`void`	fügt einen Wert am Anfang der Liste ein
`removeFirst`		`int`	entfernt den ersten Wert und gibt ihn zurück
`removeLast`		`int`	entfernt den letzten Wert und gibt ihn zurück
`clear`		`void`	entfernt alle Wert in der Liste
`isEmpty`		`boolean`	gibt zurück, ob die Liste leer ist
`get`	`int index`	`int`	gibt den Wert an der Stelle `index` zurück
`set`	`int index, int value`	`void`	ersetzt den Wert an der Stelle `index` mit `value`
`getSize`		`int`	gibt zurück, wie viele Werte die Liste enthält

Einige dieser Methoden dürfen unter gewissen Bedingungen nicht aufgerufen werden. Zum Beispiel darf `removeFirst()` nicht aufgerufen werden, wenn die Liste leer ist, oder `get()` darf nicht aufgerufen werden, wenn der gegebene Index grösser oder gleich der aktuellen Länge der Liste ist. In solchen Situationen soll sich Ihr Programm mit einer Fehlermeldung beenden. Verwenden Sie folgendes Code-Stück dafür:

```
if(condition) {
    Errors.error(message);
}
```

Ersetzen Sie *`condition`* mit der Bedingung, unter welcher das Programm beendet werden soll, und *`message`* mit einer hilfreichen Fehlermeldung. Die `Errors`-Klasse befindet sich bereits in Ihrem Projekt, aber Sie brauchen sie im Moment nicht zu verstehen.

Aufgabe 3: Executable Graph

In dieser Aufgabe verwenden wir gerichtete azyklische Graphen, um Programme zu repräsentieren. Der Programmzustand ist dabei immer durch ein Tupel $(sum, counter)$ gegeben, wobei *sum* und *counter* ganze Zahlen sind. Programmzustände werden durch `ProgramState`-Objekte modelliert, wobei `ProgramState.getSum()` (bzw. `ProgramState.getCounter()`) dem ersten Element (bzw. dem zweiten Element) des Tupels entspricht.

Eine Ausführung des Programms manipuliert den Programmzustand und das Resultat eines Programms ist gegeben durch den erreichten Programmzustand, nachdem alle Operationen im Programm ausgeführt wurden. Programme können nichtdeterministisch sein: Das heisst, für ein einzelnes Programm kann es für den gleichen Startzustand mehrere Programmausführungen geben, welche zu unterschiedlichen Resultaten führen.

Knoten in Graphen werden durch `Node`-Objekte modelliert. `Node.getSubnodes()` gibt die Kinderknoten als ein Array zurück (m ist genau dann ein Kinderknoten von n, wenn es eine ausgehende gerichtete Kante von n zu m gibt). Wir unterscheiden drei Arten von Knoten, wobei die Methode `Node.getType()` die Knotenart als `String` zurückgibt. Um ein Programm, welches durch den Knoten `n` repräsentiert wird, auszuführen, muss man den "Knoten `n` ausführen". Wir beschreiben nun die drei Knotenarten und jeweils die Ausführung der Knoten:

Aufgabe 3: Executable Graph

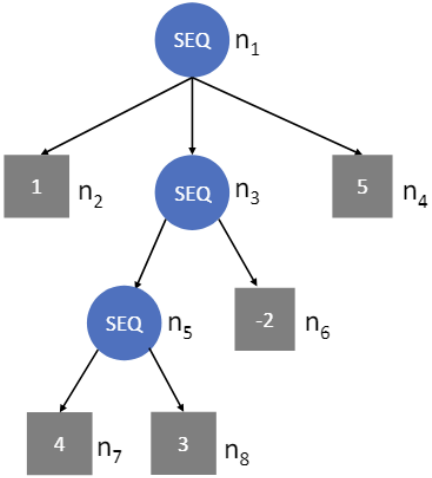
1. Additionsknoten (`Node.getType()` ist "ADD"): Solche Knoten besitzen einen Additionswert a gegeben durch `Node.getValue()` (eine ganze Zahl) und bei der Ausführung dieses Knotens wird der Programmzustand von $(sum, counter)$ zu $(sum + a, counter + 1)$ aktualisiert. Die Kinderknoten von solchen Knoten werden bei der Ausführung ignoriert.

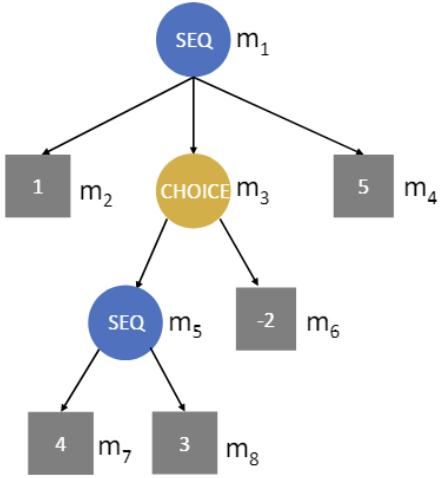
2. Sequenzknoten (`Node.getType()` ist "SEQ"): Bei der Ausführung eines Sequenzknoten `n` werden die Kinderknoten von `n` nacheinander ausgeführt. Die Reihenfolge in welcher die Kinderknoten ausgeführt werden spielt keine Rolle, da der erreichte Programmzustand für jede Reihenfolge gleich ist. `Node.getValue()` ist irrelevant.

3. Auswahlknoten (`Node.getType()` ist "CHOICE"): Bei der Ausführung eines Auswahlknoten `n` wird ein beliebiger Kinderknoten von `n` ausgewählt und ausgeführt. `Node.getValue()` ist irrelevant. Diese Knoten führen zu Nichtdeterminismus.

Sie dürfen davon ausgehen, dass Sequenz- und Auswahlknoten immer mindestens einen Kinderknoten haben, und dass es zwischen zwei Knoten immer höchstens einen Pfad gibt. Die folgende Abbildung zeigt zwei Beispielgraphen, wobei Knoten mit der Beschriftung "SEQ" (bzw. "CHOICE") Sequenzknoten (bzw. Auswahlknoten) entsprechen und die Zahlen in Additionsknoten den Additionswerten entsprechen.

Aufgabe 3: Executable Graph





Aufgabe 4: Energiespiel

In dieser Aufgabe üben Sie den Umgang mit Enums. Dafür haben Sie einen Ordner `EnergieSpiel` mit drei Klassen `GameApp`, `Game` und `Player`, sowie ein Enum `Character`. Diese sind bereits so implementiert, dass alles funktioniert. Die Klasse `Player` hat jedoch ein Feld `character` von Typ `String`. Java lässt also zu, dass in diesem Feld ein beliebiger String abgespeichert werden kann. Das Spiel hat aber eigentlich nur genau drei Möglichkeiten: `HONEST`, `TRICKSTER` oder `SORCEROR`. Das Enum `Character` mit diesen drei Optionen existiert bereits. Ändern Sie den Typ des Feldes zu `Character` und passen Sie den Code in allen drei Klassen so an, dass die Charakter-Logik überall den Typ `Character` statt `String` verwendet.

Aufgabe 5: Methoden vergleichen

Für diese Aufgabe betrachten wir folgende Methode:

```
public int T(int x, int y) {
    // body
}
```

Anstelle des Kommentars `// body` im Methodenrumpf setzen wir jeweils eine der angegebenen Anweisungsfolgen (a)–(e) ein. Vergleichen Sie für jedes unten aufgeführte Paar die Rückgabewerte für alle `x` und `y`. Antworten Sie in genau einem dieser Formate: `I; Rückgabewert: ...`, falls die Werte *immer* gleich sind; `M; Bedingung: ...; Rückgabewert: ...`, falls sie nur *manchmal* gleich sind; oder `N`, falls sie *nie* gleich sind.

Bei `M` vereinfachen Sie den gemeinsamen Rückgabewert mithilfe der angegebenen Bedingung. Aus der Bedingung `y == 9` und dem Wert `y + 1` wird beispielsweise `Rückgabewert: 10`. Falls der Wert weiterhin von Variablen abhängt, ist der vollständige Java-Ausdruck erforderlich: Für `y == 2 ? 4 : 1` sind `4` oder `1` allein falsch; `y == 9 ? y + 1 : 0` wird als `y == 9 ? 10 : 0` angegeben.

(a)
```
return x < y ? x + 1 : y - 1;
```

(b)
```
if (x >= y) {
    return y - 1;
}
return x + 1;
```

Aufgabe 6: Bonus

Aufgabe 6: Game (Bonus!)

Achtung: Diese Aufgabe gibt Bonuspunkte (siehe "Leistungskontrolle" im `www.vvz.ethz.ch`). Die Aufgabe muss eigenhändig und alleine gelöst werden. Die Abgabe erfolgt wie gewohnt per Push in Ihr Git-Repository auf dem ETH-Server. Verbindlich ist der letzte Push vor dem Abgabetermin. Auch wenn Sie vor der Deadline committen, aber nach der Deadline pushen, gilt dies als eine zu späte Abgabe. Bitte lesen Sie zusätzlich die allgemeinen Regeln.

Nachbesprechung

Aufgabe 1: Close Neighbors

Schreiben Sie ein Programm, welches für eine sortierte Folge X von `int`-Werten $(x_1, x_2, ..., x_n)$ und einen `int`-Wert `key` die drei unterschiedlichen Elemente x_a, x_b und x_c aus X zurückgibt, die dem Wert `key` am nächsten sind. Für x_a, x_b und x_c muss gelten, dass $|\texttt{key} - x_a| \leq |\texttt{key} - x_b| \leq |\texttt{key} - x_c| \leq |\texttt{key} - x_i|$ für alle $i \neq a, b, c$ und dass $x_a \neq x_b \neq x_c \neq x_a$. Wenn die drei Werte nicht eindeutig bestimmt sind, dann ist jede Lösung zugelassen, die die obige Bedingung erfüllt.

Beispiele:
Die nächsten Nachbarn für `key == 5` in $(1, 4, 5, 7, 9, 10)$ sind $5, 4, 7$.
Die nächsten Nachbarn für `key == 5` in $(1, 4, 5, 6, 9, 10)$ sind $5, 4, 6$ oder $5, 6, 4$.
Die nächsten Nachbarn für `key == 10` in $(1, 4, 5, 6, 9, 10)$ sind $10, 9, 6$.

Implementieren Sie die Berechnung in der Methode `int[] neighbor(int[] sequence, int key)`, welche sich in der Klasse `Neighbor` befindet. Die Deklaration der Methode ist bereits vorgegeben. Sie können davon ausgehen, dass das Argument `sequence` nicht `null` ist, sortiert ist, nur unterschiedliche Elemente enthält, und mindestens drei Elemente enthält. Denken Sie daran, dass der Wert `key` nicht unbedingt in der Folge X auftritt. Sie dürfen das Eingabearray `input` nicht ändern.

In der `main` Methode der Klasse `Neighbor` finden Sie die oberen Beispiele als kleine Tests, welche Beispiel-Aufrufe zur `neighbor`-Methode machen und welche Sie als Grundlage für weitere Tests verwenden können. In der Datei `NeighborTest.java` geben wir die gleichen Tests zusätzlich auch als JUnit Test zur Verfügung. Sie können diese ebenfalls nach belieben ändern. Es wird *nicht* erwartet, dass Sie für diese Aufgabe den JUnit Test verwenden.

Aufgabe 2: Loop-Invariante

1. Gegeben sind die Precondition und Postcondition für das folgende Programm

```
public int compute(int n) {
    // Precondition:  n >= 0
    int x;
    int res;

     x = 0;
    res = x;

    // Loop Invariante:
    while (x <= n) {
        res = res + x;
        x = x + 1;
    }
    // Postcondition:  res == ((n + 1) * n) / 2
    return res;
}
```

Schreiben Sie die Loop Invariante in die Datei “LoopInvariante.txt”.

Aufgabe 2: Loop-Invariante

2. Gegeben sind die Precondition und Postcondition für das folgende Programm.

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = 0;
    res = b;

    // Loop Invariante:
    while (x < a) {
        res = res - 1;
        x = x + 1;
    }
    // Postcondition:  res == b - a
    return res;
}
```

Schreiben Sie die Loop Invariante in die Datei “LoopInvariante.txt”.

Aufgabe 3: Bills

In dieser Aufgabe sollen Sie einen Teil des Systems implementieren, das für den lokalen Stromversorger die Rechnungen erstellt.

Vervollständigen Sie die `process`-Methode in der Klasse `Bills`. Die Methode hat zwei Argumente: einen `Scanner`, von dem Sie den Inhalt der Eingabedatei lesen sollen, und einen `PrintStream`, in welchen Sie die unten beschriebenen Informationen schreiben.

Ihr Programm muss nur korrekt formatierte Eingabedateien unterstützen. Ein Beispiel einer solchen Datei finden Sie im Projekt unter dem Namen "Data.txt". Exceptions im Zusammenhang mit Ein- und Ausgabe können Sie ignorieren.

Eine valide Eingabedatei enthält Zeilen, die entweder einen Tarif oder den Stromverbrauch eines Kunden beschreiben. Der Verbrauch eines Kunden ist niemals grösser als 100000 Kilowattstunden.

Eine Tarifbeschreibung hat folgendes Format:

$$\texttt{Tarif}\,␣\,n\,␣\,l_1\,␣\,p_1\,\ldots\,l_n\,␣\,p_n$$

Aufgabe 4: Minesweeper (Bonus)

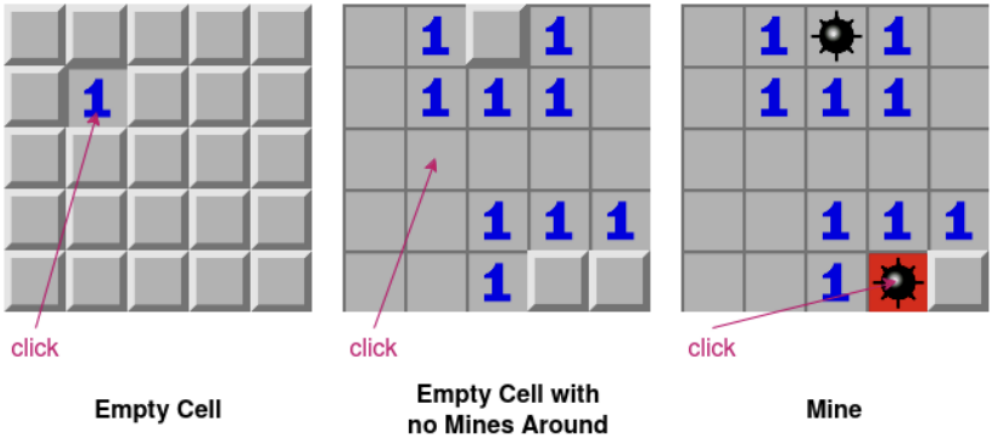


Abbildung 1: Spielbretter nach dem ersten, zweiten und dritten Klick von links nach rechts.

Achtung: Diese Aufgabe gibt Bonuspunkte (siehe "Leistungskontrolle" im `www.vvz.ethz.ch`). Die Aufgabe muss eigenhändig und alleine gelöst werden. Die Abgabe erfolgt wie gewohnt per Push in Ihr Git-Repository auf dem ETH-Server. Verbindlich ist der letzte Push vor dem Abgabetermin. Auch wenn Sie vor der Deadline committen, aber nach der Deadline pushen, gilt dies als eine zu späte Abgabe. Bitte lesen Sie zusätzlich die allgemeinen Regeln.